

PROJECT REPORT ON

Crop Yield Prediction and Resource Optimization

Using Machine Learning and Linear Programming

*An Integrated Approach of Predictive Analytics
and Optimization Techniques*

ABSTRACT

Agriculture plays a crucial role in the Indian economy, where crop productivity is significantly influenced by environmental conditions and resource management practices. This project presents an integrated approach for crop yield prediction and resource optimization using Machine Learning (ML) and Operational Research (OR) techniques.

The study utilizes a comprehensive historical dataset consisting of over 50,000 records across multiple Indian states, incorporating features such as temperature, humidity, rainfall, soil pH, solar radiation, and fertilizer requirements. Initial data preprocessing included cleaning, handling outliers using the Interquartile Range (IQR) method, and standardizing feature formats. Exploratory Data Analysis (EDA) was conducted to understand patterns, distributions, and relationships among variables, revealing strong correlations between fertilizer components and crop yield.

To address multicollinearity and prevent data leakage, redundant variables were removed and a new feature, Total Fertilizer, was engineered by combining nitrogen (N), phosphorus (P), and potassium (K) requirements. Several machine learning models, including Linear Regression, Ridge, Lasso, and Random Forest, were trained and evaluated using performance metrics such as R^2 score, RMSE, MAE, and cross-validation. Among these, Lasso Regression demonstrated the best performance with high accuracy and strong generalization capability.

The predicted yield from the ML model was then integrated into an optimization framework. Since machine learning models are typically non-linear, a linear approximation of the yield function was derived to enable the use of Linear Programming. The optimization model aims to maximize profit by determining the optimal levels of fertilizer and water usage under constraints such as budget limits, resource availability, and environmental considerations.

The final system provides actionable recommendations, including optimal fertilizer and water allocation, expected crop yield, and maximum achievable profit. This integrated ML and OR approach supports data-driven decision-making in agriculture, helping farmers and policymakers improve productivity, reduce costs, and utilize resources efficiently. Overall, this project demonstrates how combining predictive analytics with optimization techniques can solve real-world agricultural problems and contribute to sustainable farming practices

Table of Contents

1	ABSTRACT
2	INTRODUCTION
3	OBJECTIVE
4	METHODOLOGY
4.1	INTRODUCTION TO SUPERVISED LEARNING
4.2	DATA DESCRIPTION
4.2.1	Data Variables
4.3	DATA UNDERSTANDING & CLEANING
4.3.1	Checking the Column Names
4.3.2	Checking Data Information and Summary Statistics
4.3.3	Checking Numerical and Categorical Columns
4.3.4	Checking the Target Variable
4.3.5	Checking for Duplicate Records
4.3.6	Checking for Missing Values
4.3.7	Correlation Analysis
4.3.8	Renaming the Columns
4.3.9	Counting the Categorical Columns

4.3.10	Range and Scale of Numerical Features
4.3.11	Checking the Outliers Using Boxplot and IQR Methods
4.3.12	Checking the Skewness in the Columns Using Histogram
4.3.13	Outlier Handling Using Capping Method
4.4	EXPLORATORY DATA ANALYSIS (EDA).....
4.4.1	Univariate Analysis
	– Numeric Feature Analysis
	–Categorical Feature Analysis
4.4.2	Bivariate Analysis
4.4.3	Correlation Analysis
4.4.4	Trend Analysis
4.5	FEATURE ENGINEERING
4.5.1	Creating a New Feature
4.5.2	Handling Skewness using Log Transformation
4.5.3	Checking Skewness of Data
4.5.5	Detecting And Dropping Highly Correlated Features
4.5.6	Encoding Categorical Columns using One-Hot Encoding
4.5.7	Scaling The Numerical Columns
4.5.8	Checking The Variance Inflation Factor(VIF)

4.5.9	Automatic Selection of Important Features for the Model
4.6	MODEL DEVELOPMENT
4.6.1	Train-Test Split
4.6.2	Model Training
	– Linear Regression
	– Lasso Regression
	– Ridge Regression
	– Random Forest Regressor
4.7	MODEL EVALUATION
4.7.1	Performance Metrics (MAE, MSE, RMSE, R^2 , Adjusted R^2 , CV)
4.7.2	Checking Overfitting In The Fitted Model
4.7.3	Best Model Selection
4.8	OPERATIONAL RESEARCH (OPTIMIZATION MODEL)
4.8.1	Linking Machine Learning With Optimization
4.8.2	Adjustment Of Coefficients
4.8.3	Decision Variables
4.8.4	Objective Function
4.8.5	Constraints
4.8.6	Optimization Method

4.8.7	Optimal Solution
4.8.8	Maximum Profit
5	RESULTS AND DISCUSSION
6	CONCLUSION AND FUTURE WORK
7	REFERENCES

INRODUCTION

1. Background and Importance of Agriculture:

Agriculture is a fundamental sector of the Indian economy, providing livelihood to a large portion of the population. Crop productivity is highly dependent on various environmental and soil-related factors such as temperature, rainfall, humidity, and nutrient availability. However, farmers often face uncertainty in predicting crop yield due to changing climatic conditions and limited access to data-driven decision-making tools.

2. Problem Statement:

One of the major challenges in agriculture is the efficient use of resources like water and fertilizers. Overuse of these resources increases costs and harms the environment, while under-use leads to reduced crop yield. Additionally, traditional methods of yield prediction are often inaccurate and do not consider multiple influencing factors simultaneously. Therefore, there is a need for a system that can accurately predict yield and suggest optimal resource utilization.

3. Proposed Solution (ML + OR Approach):

This project proposes an integrated approach that combines Machine Learning (ML) and Operational Research (OR). Machine learning models are used to predict crop yield based on historical and environmental data, while optimization techniques such as Linear Programming are applied to determine the best allocation of resources like fertilizer and water. This dual approach ensures both accurate prediction and efficient decision-making.

4. Significance of the Project:

The developed system provides a data-driven framework for smart agriculture by helping farmers maximize profit while minimizing resource wastage. It supports better agricultural planning, improves productivity, and promotes sustainable farming practices. By integrating prediction and optimization, the project offers a practical and innovative solution to real-world agricultural problems.

OBJECTIVE

The primary objective of this project is to develop an accurate and reliable system for predicting crop yield using machine learning techniques. By utilizing historical agricultural and environmental data including temperature, rainfall, humidity, soil pH, and other climatic factors the model aims to capture the complex relationships that influence crop productivity. A key focus is to build a model that generalizes well to unseen data by addressing issues such as multicollinearity and overfitting through proper feature selection and preprocessing. The goal is to provide farmers and agricultural planners with dependable yield predictions that can support informed decision-making.

The second objective is to optimize the use of critical agricultural resources, particularly fertilizer and water, using operational research techniques. After obtaining predicted yield from the machine learning model, the project formulates a Linear Programming (LP) model to determine the optimal levels of resource allocation. The objective function is designed to maximize profit by balancing revenue from crop yield against the costs of fertilizer and water, while satisfying real-world constraints such as budget limits, resource availability, and environmental considerations. This ensures that recommendations are not only accurate but also practical and economically viable.

The final objective is to integrate machine learning and optimization into a unified decision-support system for smart agriculture. Unlike traditional approaches that focus only on prediction, this project combines predictive analytics with optimization to provide actionable recommendations. The system takes input conditions, predicts crop yield, and then suggests optimal resource usage to maximize productivity and profitability. This integrated framework aims to promote efficient resource utilization, reduce farming costs, and support sustainable agricultural practices, thereby contributing to improved agricultural planning and policy development..

METHODOLOGY

4.1 INTRODUCTION TO SUPERVISED LEARNING

What is Supervised Learning?

Supervised learning is a type of machine learning algorithm that uses a known dataset, called the training dataset, to make predictions. The term “supervised” refers to the presence of a “supervisor” or a “teacher” who guides the learning process. In this context, the supervisor is the labelled training data from which the learning algorithm learns to make predictions or decisions.

The training data consists of input-output pairs, where the input is a feature vector representing the data point, and the output is the corresponding label or target value. The goal of supervised learning is to find a function that can predict the output with high accuracy for unseen data.

Types of Supervised Learning

Supervised learning can be broadly categorized into two types:

- **Classification:** In classification tasks, the output variable is a category, such as “spam” or “not spam”, or “malignant” and “benign”. The model assigns input features to predefined classes.
- **Regression:** In regression tasks, the output variable is continuous, such as house price or temperature. The model predicts numerical values.

Training a Supervised Learning Model

The process involves the following steps:

1. **Data Collection:** Gather a large, high-quality dataset with input-output pairs.
2. **Data Preprocessing:** Handle missing values, encode categorical variables, normalize features, and reduce dimensionality.
3. **Model Selection:** Choose an appropriate algorithm based on the problem type.
4. **Training:** Fit the model by minimizing a **loss function**. Common regression losses include **MSE** and **MAE**.
5. **Validation:** Tune hyperparameters and prevent overfitting.
6. **Testing:** Evaluate performance on unseen data.

Regression Analysis

Regression analysis is used to model the relationship between a dependent variable (Y) and independent variables ($X_1, X_2, \dots, X_n$).

There are several types of regression:

1. Linear Regression

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \epsilon \quad (1)$$

Here, β_0 is the intercept, β_i are coefficients, and ϵ is the error term.

2. Ridge Regression

$$\text{Loss} = \text{MSE} + \lambda(\beta_1^2 + \beta_2^2 + \dots + \beta_n^2) \quad (2)$$

Uses L_2 regularization to reduce overfitting.

3. Lasso Regression

$$\text{Loss} = \text{MSE} + \lambda(|\beta_1| + |\beta_2| + \dots + |\beta_n|) \quad (3)$$

Uses L_1 regularization and performs feature selection.

4. Random Forest Regressor

Random Forest builds multiple decision trees and combines their outputs. It captures non-linear relationships and is robust but requires tuning.

All models aim to minimize loss and improve performance. In this project, models were evaluated using R^2 , MSE, RMSE, and MAE with cross-validation.

4.2 DATA DESCRIPTION

The dataset used in this project was compiled from publicly available agricultural and environmental data sources, including resources from the Ministry of Agriculture Farmers Welfare, Government of India, the India Meteorological Department (IMD), and the Open Government Data Platform India (data.gov.in). The dataset contains crop production, climatic, and soil-related information across multiple states in India.

The dataset includes important variables such as year of cultivation, land area, temperature, humidity, rainfall, soil pH, wind speed, solar radiation, and fertilizer requirements (Nitrogen, Phosphorus, and Potassium). These variables play a crucial role in determining crop productivity and agricultural performance.

By analyzing these features, the project aims to accurately predict crop yield using machine learning techniques and further optimize resource usage such as fertilizer and water using op-

erational research methods. The dataset covers multiple crops and states, providing a diverse and realistic representation of agricultural conditions in India.

4.2.1 Data Variables:

- **Year:** Represents the year of crop cultivation.
- **State Name:** Indicates the state where the crop is grown.
- **District Name:** Specifies the district of cultivation.
- **Crop:** Type of crop cultivated (e.g., rice, maize, chickpea, cotton).
- **Area_ha:** Total cultivated land area in hectares.
- **Yield_kg_per_ha (Target Variable):** Crop yield per hectare.
- **N_req_kg_per_ha:** Nitrogen fertilizer requirement per hectare.
- **P_req_kg_per_ha:** Phosphorus fertilizer requirement per hectare.
- **K_req_kg_per_ha:** Potassium fertilizer requirement per hectare.
- **Total_N_kg / Total_P_kg / Total_K_kg:** Total fertilizer usage based on area.
- **Temperature_C:** Average temperature during crop growth.
- **Humidity_%:** Relative humidity percentage.
- **pH:** Soil pH value indicating acidity/alkalinity.
- **Rainfall_mm:** Total rainfall received (mm).
- **Wind_Speed_m_s:** Wind speed in meters per second.
- **Solar_Radiation_MJ_m²_day:** Solar radiation received per day.

Summary Statistics and Initial Observations

The dataset consists of approximately 50,000 observations and includes a wide range of agricultural, climatic, and soil-related variables, making it suitable for predictive modeling and optimization.

The Year variable ranges from around 1966 to 2017, indicating long-term agricultural trends. The dataset includes data from multiple states and districts across India, ensuring diversity in environmental conditions.

The Area_ha variable shows significant variation, ranging from small-scale farms to very large agricultural lands. This variability reflects real-world farming conditions where landholding sizes differ across regions.

The Yield_kg_per_ha, which is the target variable, varies widely across crops and regions. Initially, the dataset exhibited high variability and the presence of outliers, which were handled using the Interquartile Range (IQR) method. After treatment, the distribution became more stable and suitable for modeling.

The fertilizer-related variables (N, P, K requirements) show strong correlation with crop yield. However, these variables were carefully handled during modeling to avoid data leakage, as they are directly related to yield.

Climatic variables such as Temperature, Rainfall, and Humidity show moderate influence on crop yield, indicating that environmental conditions play an important role in agricultural productivity.

The Rainfall variable ranges approximately from 600 mm to 1200 mm, while Temperature varies between 20°C and 28°C, representing typical agricultural climate conditions in India.

The Humidity and Solar Radiation variables show relatively stable distributions, suggesting consistent environmental patterns across regions.

4.3 DATA UNDERSTANDING AND CLEANING

4.3.1 Checking the Column Names

In this step, the column names of the dataset were carefully examined to ensure they are clear, consistent, and suitable for analysis and modeling. Initially, the column names were inspected using Python commands such as `df.columns` to display all feature names and verify their structure.

During this process, it was observed that some column names contained spaces, special characters, or inconsistent formatting, which can lead to errors while coding and model building. To address this, the column names were cleaned and standardized by removing unnecessary spaces, replacing special characters with underscores, and converting all names into a uniform format using commands like:

```
df.columns = df.columns.str.strip()
df.columns = df.columns.str.replace(' ', '_')
```

This step ensures that all column names are easy to reference, reduces the chances of syntax errors, and improves the overall readability and maintainability of the code throughout the project.

4.3.2 Checking Data Information and Summary Statistics

In this step, the overall structure and basic statistical properties of the dataset were analyzed using functions such as `.info()` and `.describe()`. The `.info()` function was used to examine the number of observations, column names, data types, and non-null values, which helped confirm that the dataset was complete and correctly formatted.

The `.describe()` function provided summary statistics for numerical features, including mean, median, standard deviation, minimum, and maximum values. From this analysis, it was observed that the dataset contains a large number of records with no missing values, and the numerical variables exhibit considerable variation across different features such as yield, rainfall, and temperature.

This step helped in understanding the distribution and scale of the data, identifying potential anomalies, and forming a strong foundation for further preprocessing and modeling.

4.3.3 Checking Numerical and Categorical Columns

In this step, the dataset was analyzed to distinguish between numerical and categorical features, which is essential for applying appropriate preprocessing techniques. Numerical columns include continuous and discrete values such as temperature, rainfall, yield, and area, while categorical columns include variables like state name and crop type.

To identify these categories, `df.select_dtypes(include=['int64', 'float64']).columns` were used to extract numerical columns, and `df.select_dtypes(include=['object']).columns` were used to identify categorical columns.

This classification helps in deciding suitable transformations, such as scaling for numerical features and encoding techniques like one-hot encoding for categorical variables.

As a result, the dataset was clearly structured into numerical and categorical groups, enabling efficient preprocessing and improving model performance.

4.3.4 Checking the Target Variable

In this step, the target variable of the dataset was identified and analyzed, which is a crucial part of any supervised machine learning problem. In this project, the target variable is **Yield_kg_per_ha**, representing the crop yield per hectare.

The distribution and range of this variable were examined to understand its variability across different observations. It was observed that the yield values vary significantly depending on factors such as crop type, environmental conditions, and fertilizer usage.

Additionally, the presence of skewness and potential outliers in the target variable was identified, which required careful preprocessing in later stages.

Understanding the behavior of the target variable is important because it directly influences model performance and helps in selecting appropriate modeling techniques and evaluation metrics.

4.3.5 Checking for Duplicate Records

In this step, the dataset was examined to identify any duplicate rows using the Python command `df.duplicated().sum()`. This function helps in detecting repeated records that may negatively impact the accuracy of the model.

Duplicate data can lead to biased learning, as the model may give more importance to repeated observations. Upon performing this check, it was observed that the dataset contained zero duplicate records, indicating that all entries were unique.

Therefore, no further action such as removing duplicates was required. This step ensured the integrity and reliability of the dataset before proceeding to further preprocessing and analysis.

```
# checking the missing values:  
print(df.isnull().sum())
```

```
Dist Code      0  
Year           0  
State Code     0  
State Name     0  
Dist Name      0  
Crop           0  
Area_ha        0  
Yield_kg_per_ha 0  
N_req_kg_per_ha 0  
P_req_kg_per_ha 0  
K_req_kg_per_ha 0  
Total_N_kg     0  
Total_P_kg     0  
Total_K_kg     0  
Temperature_C  0  
Humidity_%     0  
pH             0  
Rainfall_mm    0  
Wind_Speed_m_s 0  
Solar_Radiation_MJ_m2_day 0  
dtype: int64
```

4.3.6 Checking for Missing Values

In this step, the dataset was analyzed to identify any missing or null values that could affect the performance of the machine learning model. The command `df.isnull().sum()` was used to check the number of missing values present in each column.

Upon examination, it was observed that all columns contained zero missing values, indicating that the dataset is complete and does not require any imputation or data filling techniques. This is an important advantage, as missing values can lead to biased results or reduce model accuracy if not handled properly.

Therefore, since no missing values were found, the dataset was considered clean and ready for further preprocessing and analysis.

```
# checking the duplicates in the dataset:  
print(df.duplicated().sum())
```

0

4.3.7 Correlation Analysis

In this step, correlation analysis was performed to understand the relationship between different numerical variables in the dataset, especially their influence on the target variable (**Yield_kg_per_ha**).

This was done using the following command:

```
correlation = df.corr(numeric_only=True)  
print(correlation)
```

The correlation matrix provides values between -1 and $+1$, where values closer to $+1$ indicate a strong positive relationship, values closer to -1 indicate a strong negative relationship, and values near 0 indicate little or no relationship.

From the analysis, it was observed that the fertilizer-related variables (**N_req_kg_per_ha**, **P_req_kg_per_ha**, **K_req_kg_per_ha**) showed a very high positive correlation with the yield, indicating their strong influence on crop production.

Additionally, variables such as rainfall and humidity showed moderate positive relationships with yield, while temperature had a relatively weaker effect.

This step was important for identifying key influencing features and also helped in detecting multicollinearity and potential data leakage, which were considered in later stages of model building.

4.3.8 Renaming the Columns (Removing Spaces and Symbols)

In this step, the column names were standardized by removing spaces and special characters to ensure consistency and ease of use during analysis and modeling. Some columns originally

contained spaces and symbols like % which can cause issues while writing code or accessing columns in Python. Therefore, these were replaced with underscores and meaningful text formats.

This improves readability, avoids syntax errors, and ensures smooth execution of data processing and machine learning operations.

Python Code Used:

```
# Remove spaces
df.columns = df.columns.str.replace(" ", "_")
# Replace '%' symbol
df.columns = df.columns.str.replace("%", "_percent")
```

Result:

- Column names became clean and consistent.
- Example: Humidity_% → Humidity_percent
- Improved compatibility with Python and machine learning models.

4.3.9 Counting the Categorical Columns

In this step, the number and type of categorical variables present in the dataset were identified using Python commands. This was done by selecting columns with object data types, which represent non-numerical features such as State Name, District Name, and Crop Type.

By using functions like `select_dtypes(include='object')`, the categorical columns were extracted and their count was determined. It was observed that the dataset contains a few important categorical variables that provide qualitative information about location and crop type.

Identifying and counting these categorical columns is important because they require special preprocessing techniques, such as encoding, before being used in machine learning models.

This step helped in planning appropriate transformations for these variables during the preprocessing stage.

4.3.10 Range and Scale of Numerical Features

In this step, the range and scale of numerical features in the dataset were analyzed to understand the variability and magnitude of different variables. This was done by examining the minimum and maximum values of each numerical column.

It was observed that there is a significant variation in the scale of features. For example, variables like **Area_ha** and total fertilizer values have very large ranges, while variables such as **pH**, temperature, and wind speed have relatively smaller and more stable ranges. Similarly, the target variable **Yield_kg_per_ha** also shows a wide range of values across different observations.

Such differences in scale can affect the performance of machine learning models, especially those sensitive to feature magnitude. This analysis highlighted the importance of applying appropriate scaling or normalization techniques during preprocessing to ensure that all features contribute fairly to the model.

```
# range and scale of the numeric features:  
print(df.min(numeric_only=True))  
print(df.max(numeric_only=True))
```

4.3.11 Checking the Outliers Using Boxplot and IQR Methods

In this step, the dataset was analyzed to identify the presence of outliers, which are extreme values that deviate significantly from the majority of the data. Outliers can negatively impact the performance of machine learning models by distorting patterns and influencing model predictions. Therefore, detecting and understanding outliers is an important part of data preprocessing.

In this project, two effective methods were used to detect outliers: **Boxplot Visualization** and the **Interquartile Range (IQR) Method**.

1. Boxplot Method

The boxplot is a graphical representation used to visualize the distribution of numerical data and identify outliers. It displays key statistical measures such as the median, quartiles (Q_1 and Q_3), and the spread of the data. In a boxplot, any data points that lie outside the “whiskers” (the range defined by $Q_1 - 1.5 \times IQR$ and $Q_3 + 1.5 \times IQR$) are considered outliers.

In this project, boxplots were generated for important numerical features such as Yield, fertilizer variables (N, P, K), temperature, humidity, pH, and rainfall. From these visualizations, it was observed that variables like **Yield_kg_per_ha** and fertilizer-related features showed significant outliers, indicating the presence of extreme agricultural values in certain observations.

On the other hand, variables such as temperature and pH showed relatively stable distributions with minimal or no outliers. The boxplot method provided a quick and intuitive way to visually inspect the spread and variability of the data.

2. Interquartile Range (IQR) Method

The Interquartile Range (IQR) method is a statistical technique used to detect outliers based on the spread of the middle 50% of the data. It is calculated as:

$$IQR = Q_3 - Q_1 \quad (4)$$

where:

- Q_1 (25th percentile) represents the lower quartile
- Q_3 (75th percentile) represents the upper quartile

Using the IQR, the lower and upper bounds for detecting outliers are defined as:

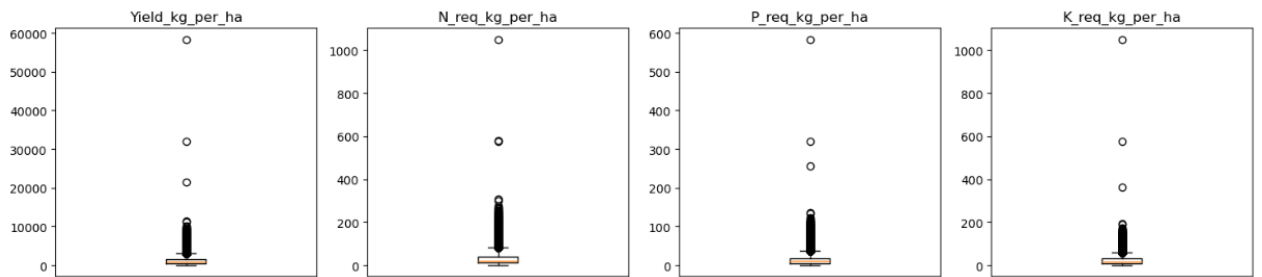
$$\text{Lower Bound} = Q_1 - 1.5 \times IQR \quad (5)$$

$$\text{Upper Bound} = Q_3 + 1.5 \times IQR \quad (6)$$

Any data points falling outside these bounds are considered outliers.

In this project, the IQR method was applied to multiple numerical columns. The results showed that Yield and fertilizer-related features had a considerable number of outliers, while other environmental variables such as rainfall, temperature, and humidity had very few or no outliers.

This confirmed the observations made from the boxplots and provided a more precise quantitative measure of outliers in each feature.



Conclusion

By combining both boxplot visualization and the IQR method, outliers were effectively identified in the dataset. The boxplot provided a visual understanding, while the IQR method offered a mathematical approach to detect extreme values.

4.3.12 Checking the Skewness in the Columns Using Histogram

In this step, the distribution of numerical features was analyzed using histograms to identify the presence of skewness in the data. Skewness refers to the asymmetry in the distribution of a variable, which can affect the performance of machine learning models if not properly handled. Histograms were plotted for key numerical variables such as **Yield_kg_per_ha**, **rain-fall**, **temperature**, **humidity**, **fertilizer values**, and area to visually examine their distribution patterns.

From the analysis, it was observed that several variables, especially **Yield_kg_per_ha** and fertilizer-related features, exhibited right-skewed distributions, where a majority of the values are concentrated on the lower side and a few extreme values extend towards the higher side. This indicates the presence of high-value observations or outliers in the dataset. On the other hand, some variables like temperature and pH showed relatively symmetric distributions, indicating a more balanced spread of values.

Identifying skewness is important because highly skewed data can negatively impact model accuracy and lead to biased predictions. This analysis helped in understanding the underlying data distribution and supported the need for applying transformations or outlier handling techniques in later stages of preprocessing, ensuring better model performance and stability.

4.3.13 Outlier Handling Using Capping Method

After identifying outliers using boxplots and the Interquartile Range (IQR) method, the next step was to handle these extreme values to improve data quality and model performance. In this project, the capping method (also known as winsorization) was used to treat outliers. Instead of removing outliers, which could lead to loss of important information, the capping method limits extreme values to a specified threshold, thereby reducing their impact while preserving the overall dataset.

The capping limits were determined using the Interquartile Range (IQR) approach. For each numerical feature, the lower and upper bounds were calculated as:

$$\text{Lower Bound} = Q_1 - 1.5 \times IQR \quad (7)$$

$$\text{Upper Bound} = Q_3 + 1.5 \times IQR \quad (8)$$

Any values below the lower bound were replaced with the lower bound, and values above the upper bound were replaced with the upper bound. This approach ensures that extreme values are brought within a reasonable range without completely discarding them.

In this project, capping was applied to features such as **Yield_kg_per_ha**, fertilizer variables,

and area, where significant outliers were observed. After applying the capping method, the distributions of these variables became more stable and less skewed, which helped in improving the performance and reliability of the machine learning models.

The advantage of using the capping method is that it maintains the size of the dataset while minimizing the influence of extreme values. This makes it particularly useful in real-world datasets like agricultural data, where extreme values may represent rare but valid conditions. Overall, this step played a crucial role in enhancing data consistency and ensuring better model generalization.

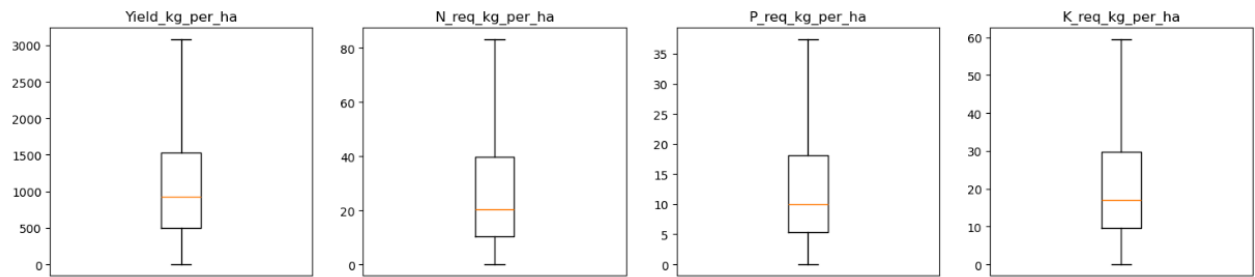
```
# Handling the outliers in columns using the cap-method:
cols = ['N_req_kg_per_ha', 'P_req_kg_per_ha', 'K_req_kg_per_ha', 'Yield_kg_per_ha']
for col in cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    df[col] = df[col].clip(lower, upper)
columns=["Yield_kg_per_ha", "N_req_kg_per_ha", "P_req_kg_per_ha", "K_req_kg_per_ha",
        "Temperature_C", "Humidity_percent", "pH", "Rainfall_mm", "Wind_Speed_m_s",
        "Solar_Radiation_MJ_m2_day"]
for col in columns:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    outliers = df[(df[col] < lower) | (df[col] > upper)]
    | print(f"{col} → {len(outliers)} outliers")
```

output:

```
Yield_kg_per_ha → 0 outliers
N_req_kg_per_ha → 0 outliers
P_req_kg_per_ha → 0 outliers
K_req_kg_per_ha → 0 outliers
Temperature_C → 0 outliers
Humidity_percent → 0 outliers
pH → 0 outliers
Rainfall_mm → 0 outliers
Wind_Speed_m_s → 0 outliers
Solar_Radiation_MJ_m2_day → 0 outliers
```

checking outlier after appling cap method



4.4 EXPLORATORY DATA ANALYSIS (EDA)

Exploratory Data Analysis (EDA) is a crucial step in the data science process that helps in understanding the structure, distribution, and relationships within the dataset before building machine learning models. In this project, EDA was performed using statistical summaries and visualizations such as histograms, boxplots, scatter plots, and correlation heatmaps.

The goal was to identify patterns, detect anomalies, and gain insights into the factors affecting crop yield.

4.4.1 Univariate Analysis

Univariate analysis focuses on examining each variable individually to understand its distribution, central tendency, spread, and overall behavior. In this project, univariate analysis was performed using visualization techniques such as histograms, distribution plots, and bar plots to gain insights into both numerical and categorical variables.

Numerical Features Analysis (Histogram / Histplot)

Histograms (histplots) were used to analyze the distribution of numerical variables such as:

- **Yield_kg_per_ha**
- **Temperature_C**
- **Rainfall_mm**
- **Humidity_percent**
- **pH**
- **Total_Fertilizer**

A histogram represents the frequency distribution of data by dividing it into bins and showing how often values fall within each range.

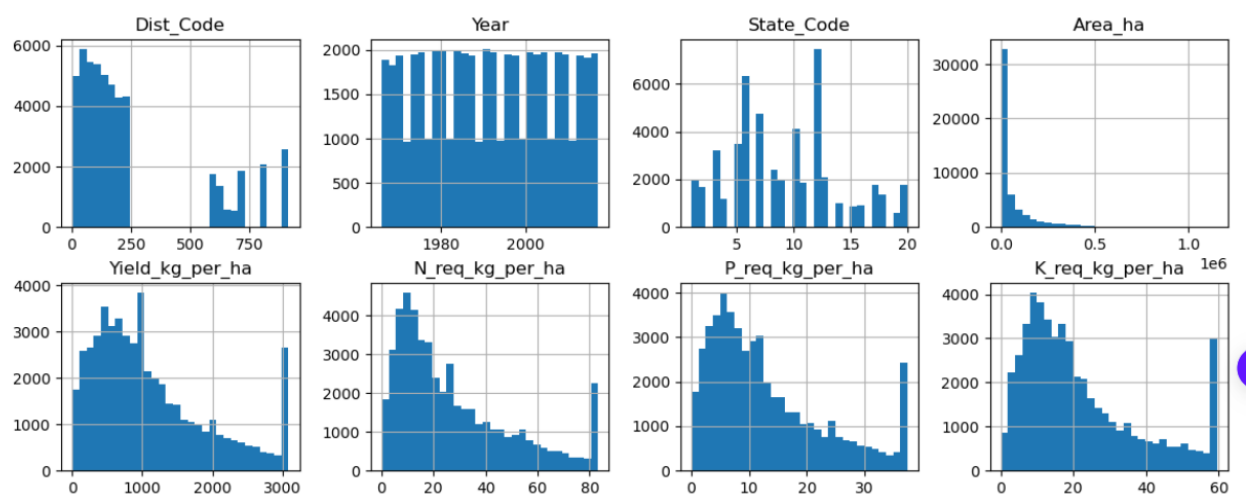
Observations:

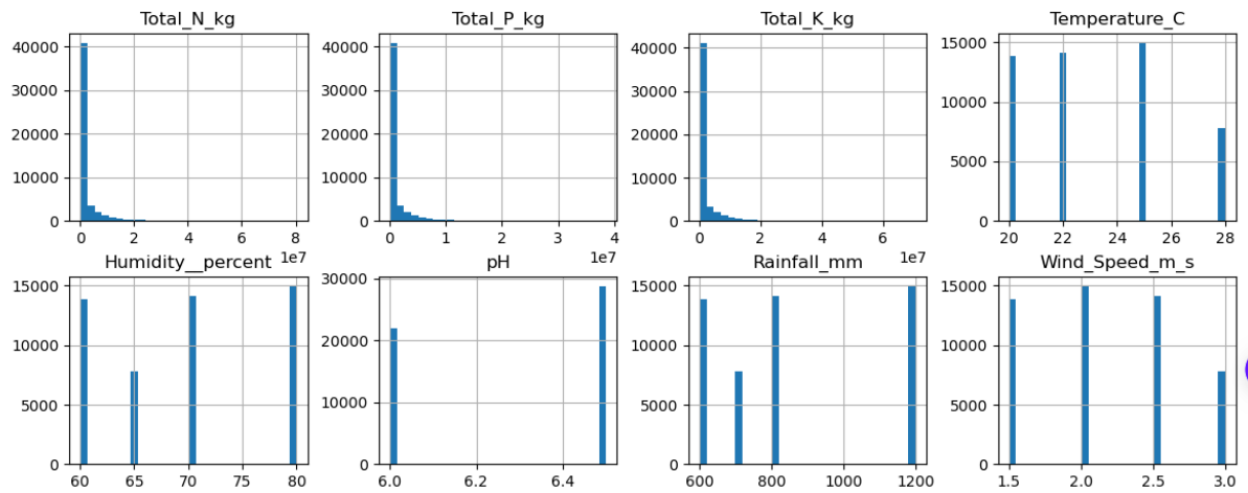
- The **Yield** variable showed a right-skewed distribution, where most values are concentrated at lower and medium ranges, while a few observations have very high yield values.
- The **Total_Fertilizer** variable also showed skewness and the presence of extreme values, indicating variability in fertilizer usage across different regions.
- Environmental variables such as **Temperature** and **Humidity** showed relatively normal distributions, suggesting stable climatic conditions across observations.
- **Rainfall** exhibited moderate variation, indicating differences in water availability across regions.

Result:

- Helped in identifying skewness and variability.
- Supported the need for outlier handling and transformation.

```
# hist plot for the all numerical features:  
df.hist(figsize=(14,14), bins=30)  
plt.show()
```





Target Variable Distribution (Distribution Plot)

A distribution plot (distplot or KDE plot) was used to specifically analyze the behavior of the target variable:

⇒ **Yield_kg_per_ha**

Observations:

- The distribution of yield was positively skewed (right-skewed)
- Most observations are concentrated around moderate yield values
- A few extreme values represent very high productivity cases

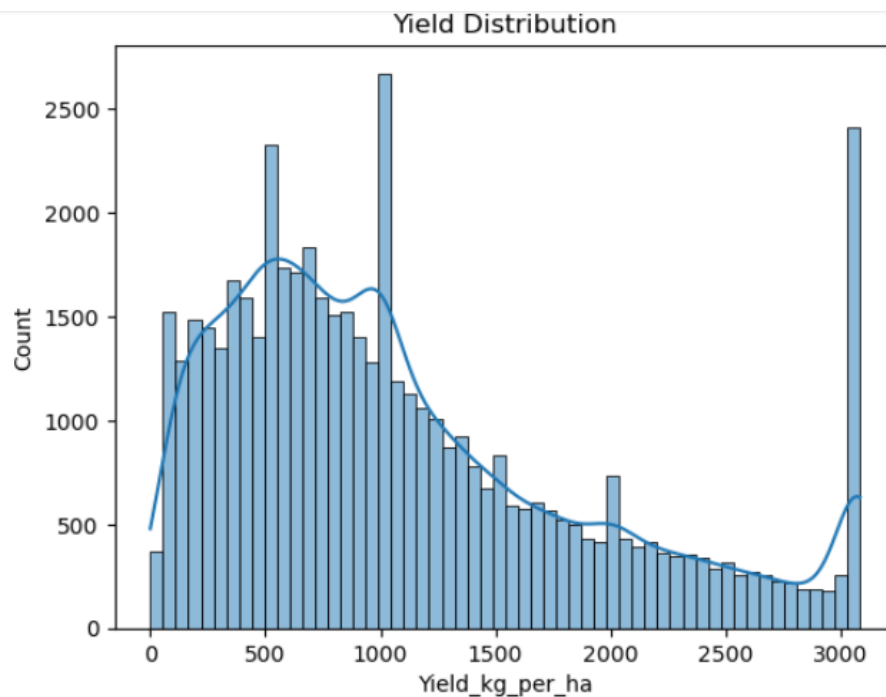
Interpretation:

- The skewness indicates that high-yield cases are less frequent but significant
- This can affect model performance if not handled properly

Result:

- Justified preprocessing steps such as outlier treatment and normalization
- Helped in understanding the nature of the prediction target

```
# distribution plot for the target variable(better than hist plot):
sns.histplot(df["Yield_kg_per_ha"],kde=True)
plt.title("Yield Distribution")
plt.show()
```



Categorical Features Analysis (Bar Plot)

Bar plots were used to analyze categorical variables such as:

- State Name
- Crop Type

A bar plot shows the frequency (count) of each category in the dataset.

State-wise Distribution

Observations:

- Some states such as Uttar Pradesh, Madhya Pradesh, and Maharashtra had a higher number of records.
- Other states had comparatively fewer observations.

Interpretation:

- Indicates uneven distribution of data across regions.
- Reflects real-world agricultural activity differences.

Crop-wise Distribution

Observations:

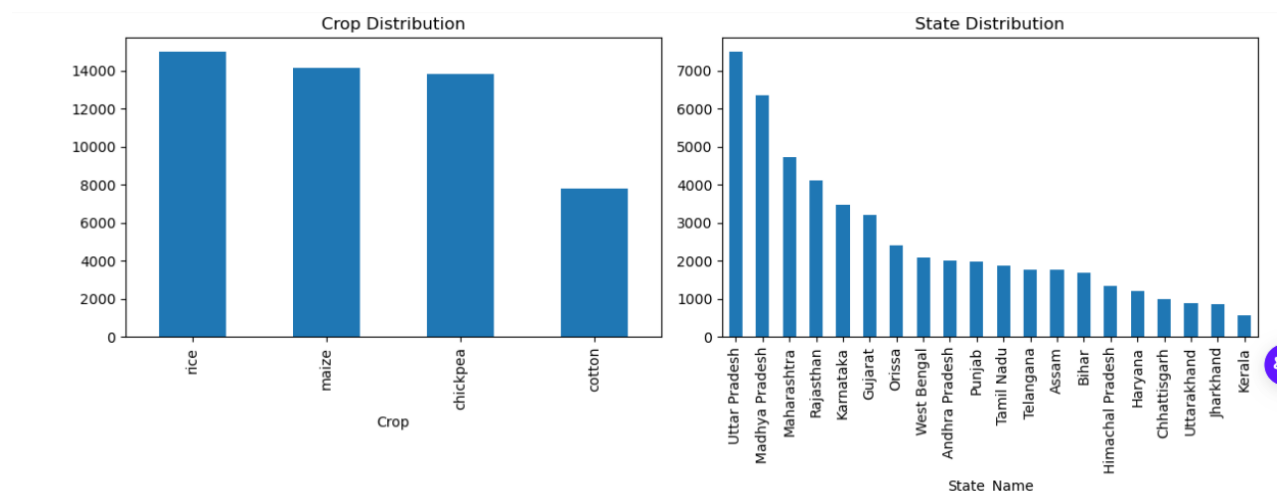
- Crops like rice, maize, cotton, and chickpea were prominently represented.
- Some crops had fewer samples.

Interpretation:

- Shows diversity in crop types.
- Helps in understanding crop-specific yield patterns.

Bar plot for categorical variables

```
# Bar plot for categorical variable:
plt.figure(figsize=(12,5))
# Plot 1: Crop
plt.subplot(1,2,1)
df["Crop"].value_counts().plot(kind="bar")
plt.title("Crop Distribution")
# Plot 2: State
plt.subplot(1,2,2)
df["State_Name"].value_counts().plot(kind="bar")
plt.title("State Distribution")
plt.tight_layout()
plt.show()
```



4.4.2 Bivariate Analysis

Bivariate analysis is used to study the relationship between two variables, particularly between input features and the target variable. In this project, scatter plots were used to visualize how different environmental and agricultural factors influence crop yield. This analysis helps in identifying patterns, trends, and the strength of relationships between variables.

Yield vs Rainfall (Scatter Plot)

A scatter plot was used to analyze the relationship between **Rainfall_mm** and **Yield_kg_per_ha**.

Observations:

- A positive relationship was observed between rainfall and crop yield.
- As rainfall increases, yield tends to increase up to a certain level.
- After a threshold, the increase in yield becomes less significant, indicating diminishing returns.
- Some scattered points indicate variability due to other influencing factors like soil quality and fertilizer usage.

Interpretation:

- Adequate rainfall is essential for crop growth, but excessive rainfall does not always lead to higher yield.
- The relationship is non-linear and moderately strong.

Yield vs Temperature (Scatter Plot)

A scatter plot was used to examine the relationship between **Temperature_C** and **Yield_kg_per_ha**.

Observations:

- The relationship between temperature and yield is moderate and scattered.
- Yield increases within an optimal temperature range but decreases outside this range.
- No strong linear pattern is observed, indicating temperature alone is not a dominant factor.

Interpretation:

- Crop growth is sensitive to temperature, but it works in combination with other factors like rainfall and soil conditions.
- The relationship is non-linear and weaker compared to fertilizer impact.

Yield vs Fertilizer (Scatter Plot)

A scatter plot showing relationship between **Total_Fertilizer** and **Yield_kg_per_ha**.

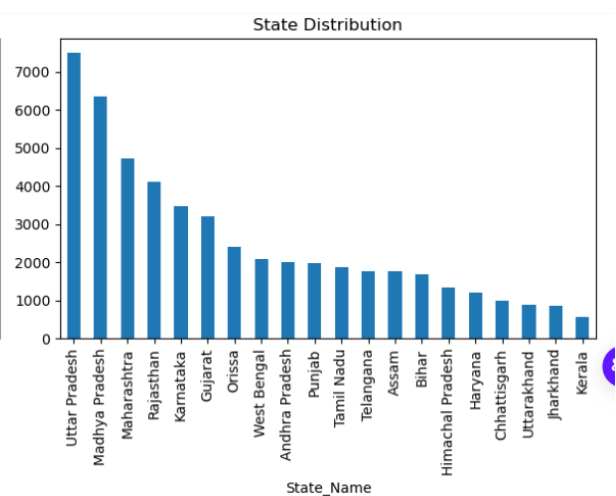
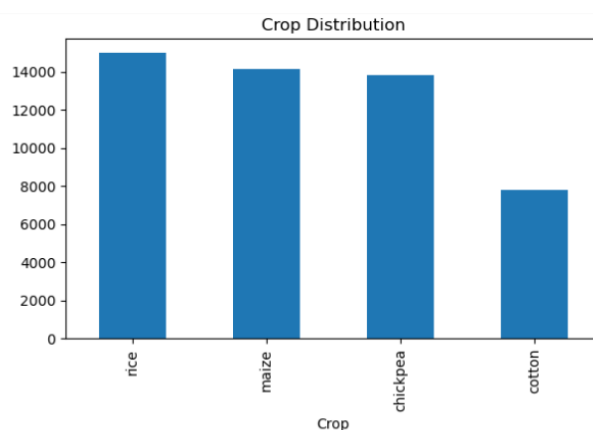
Observations:

- A strong positive relationship was observed between fertilizer usage and yield.
- As fertilizer increases, crop yield increases significantly.
- The data points are closely aligned, indicating a strong correlation.

Interpretation:

- Fertilizer plays a major role in improving crop productivity.
- Extremely high fertilizer usage may not always be efficient and can lead to diminishing returns or environmental impact.

```
# Bivariate Analysis:
# scatter plot(numeric vs numeric)
plt.figure(figsize=(15, 4))
plt.subplot(1, 3, 1)
plt.scatter(df['Rainfall_mm'], df['Yield_kg_per_ha'])
plt.title("Rainfall vs Yield")
plt.subplot(1, 3, 2)
plt.scatter(df['Temperature_C'], df['Yield_kg_per_ha'])
plt.title("Temperature vs Yield")
plt.subplot(1, 3, 3)
plt.scatter(df['N_req_kg_per_ha'], df['Yield_kg_per_ha'])
plt.title("Fertilizer vs Yield")
plt.tight_layout()
plt.show()
```



4.4.3 Correlation Analysis

Correlation analysis is used to measure the strength and direction of the relationship between numerical variables in the dataset. In this project, a correlation matrix was generated and visualized using a heatmap to understand how different features are related to each other, especially their relationship with the target variable **Yield kg per ha**.

The correlation coefficient ranges from -1 to $+1$, where values close to $+1$ indicate a strong positive relationship, values close to -1 indicate a strong negative relationship, and values near 0 indicate little or no relationship.

Key Observations

- **Yield and Fertilizer Variables:**

A very strong positive correlation was observed between Yield and Nitrogen (N), Phosphorus (P), and Potassium (K).

Interpretation: These variables are directly related to crop growth and productivity. However, such high correlation indicates multicollinearity.

- **Important Insight:**

Fertilizer variables are closely linked to yield and may lead to data leakage, which can cause the model to overfit.

- **Yield and Rainfall:**

A moderate positive correlation was observed between rainfall and yield.

Interpretation: Rainfall contributes to crop growth, but its impact is not as strong as fertilizer. This indicates environmental influence on yield.

- **Yield and Temperature:**

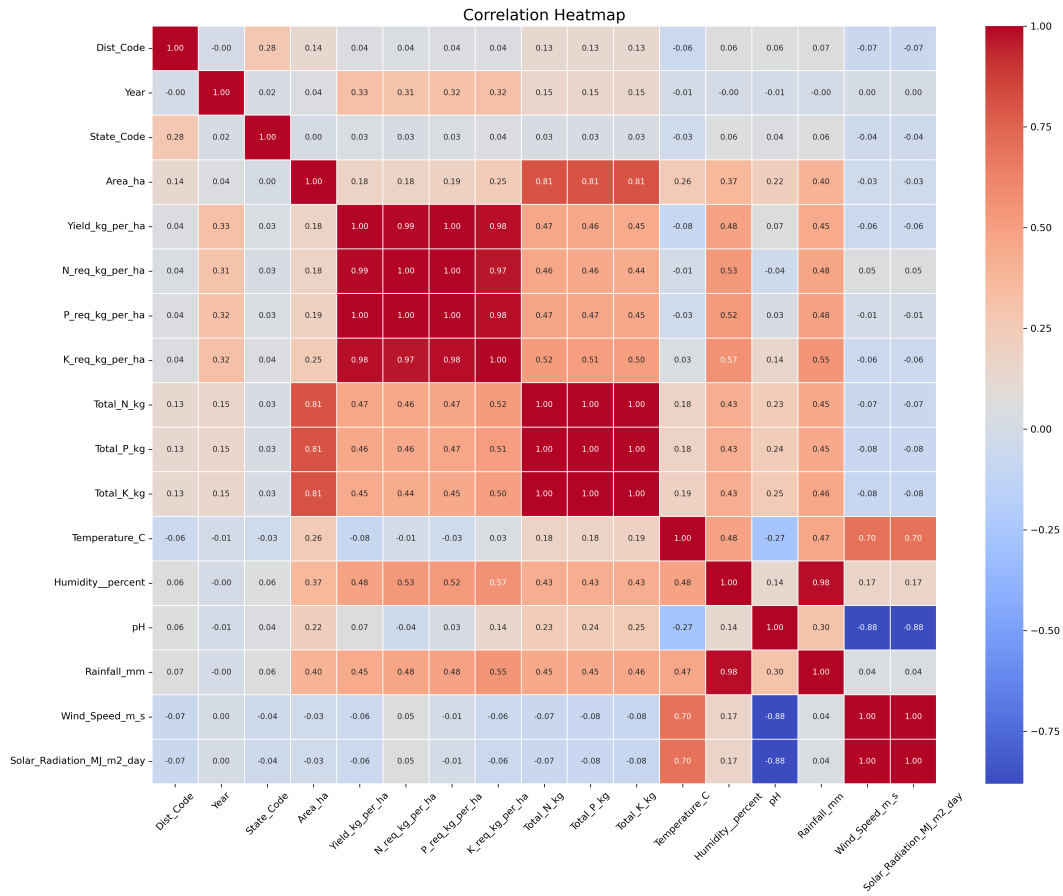
A moderate to weak correlation was observed between temperature and yield.

Interpretation: Temperature affects crop growth but only within an optimal range, and its impact is less dominant.

- **Yield and Humidity:**

A moderate relationship was observed between humidity and yield.

Interpretation: Humidity supports plant growth but does not independently determine yield.



Multicollinearity Issue

The correlation matrix revealed that fertilizer variables (N, P, K) are highly correlated with each other and also with yield.

Problem:

- Reduces model interpretability
- Causes instability in linear models
- Leads to overfitting

Action Taken As The

To address this issue, individual fertilizer variables were combined into a single feature:

$$\text{Total_Fertilizer} = N + P + K \quad (9)$$

This reduced redundancy and improved model performance.

Checking Feature Distribution using Pairplot

In this step, a pairplot was used to analyze the distribution of individual features as well as the relationships between multiple numerical variables simultaneously. A pairplot provides a combination of histograms (for individual feature distributions) and scatter plots (for pairwise relationships), making it a powerful tool for exploratory data analysis.

In this project, pairplots were generated for key numerical variables such as **Yield_kg_per_ha**, **Rainfall_mm**, **Temperature_C**, **Humidity_%**, **pH**, and **Total_Fertilizer**.

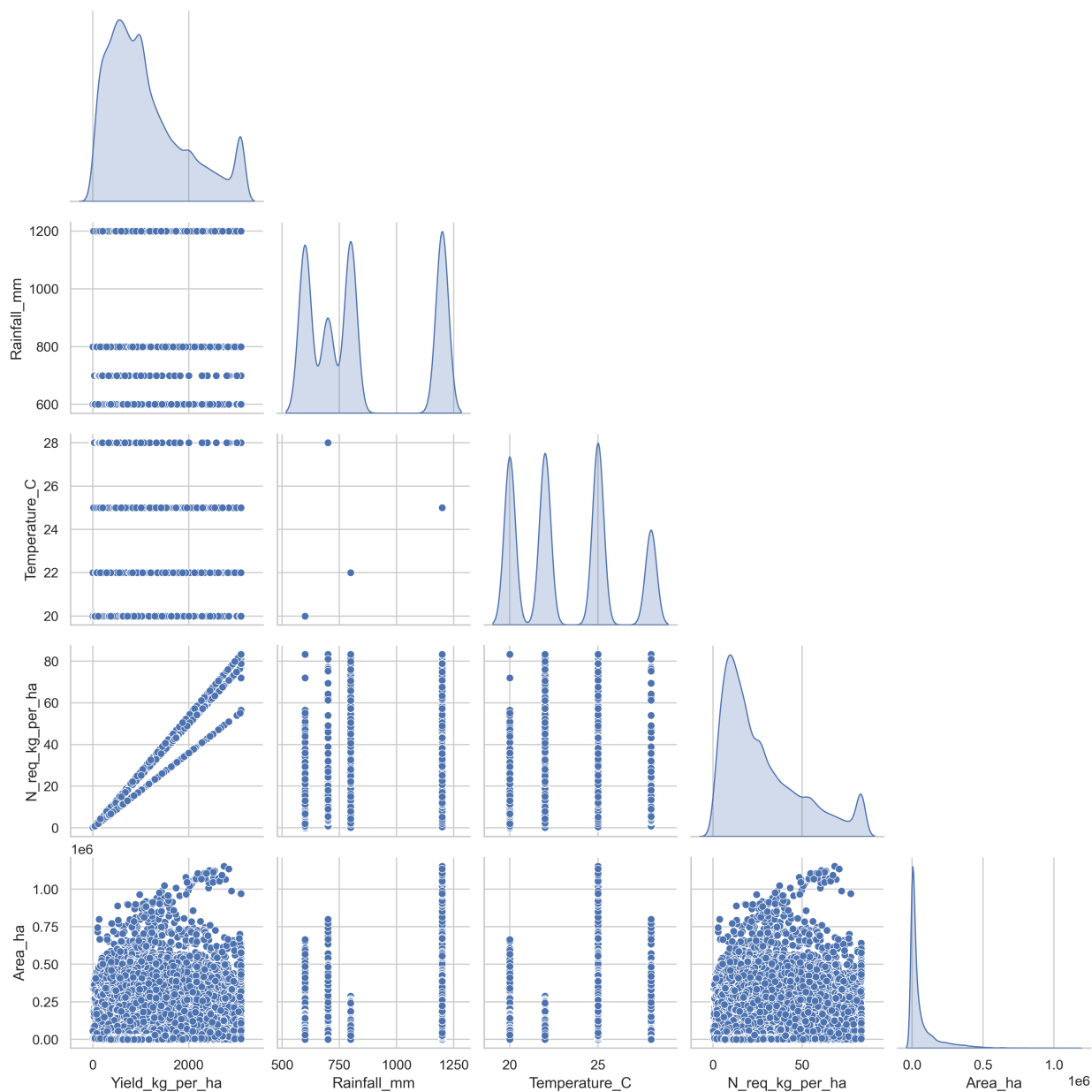
Observations:

- The diagonal plots (histograms) showed the distribution of each feature, where variables like yield and fertilizer exhibited slight skewness, while temperature and humidity appeared more normally distributed.
- The scatter plots revealed relationships between variables, confirming that yield has a strong positive relationship with fertilizer and a moderate relationship with rainfall.
- Some plots showed clusters and patterns, indicating that certain combinations of features lead to similar yield levels.
- No extreme irregular patterns were observed, suggesting that the dataset is well-structured after preprocessing.

Interpretation:

- Pairplot helped in visually confirming earlier findings from correlation and bivariate analysis.
- It provided insights into feature interactions and dependencies, which are important for model building.
- It also helped in identifying whether relationships are linear or non-linear.

```
# check feature distribution:
important_cols = [ "Yield_kg_per_ha", "Rainfall_mm", "Temperature_C",
                  "N_req_kg_per_ha", "Area_ha" ]
sns.pairplot(df[important_cols], corner=True, diag_kind='kde')
plt.savefig("pair_plots.png", dpi=300)
plt.show()
```



4.4.4 Trend Analysis

Trend analysis was performed to understand how crop yield varies across different categories such as crop type, state, and year. This helps in identifying patterns and variations in agricultural productivity.

Crop-wise Yield Trend

The crop-wise trend analysis shows that different crops have varying yield levels. Crops like rice and maize generally exhibit higher yield compared to crops like cotton or chickpea. This variation is due to differences in crop characteristics, soil requirements, and farming practices.

Insight: Crop type plays a significant role in determining yield, and crop-specific strategies are important for improving productivity.

State-wise Yield Trend

The state-wise analysis indicates that crop yield varies significantly across different states. Some states show consistently higher yields due to better irrigation facilities, favorable climatic conditions, and advanced agricultural practices, while others show lower productivity.

Insight: Geographical and environmental factors strongly influence crop yield, highlighting the importance of region-specific agricultural planning.

Year-wise Yield Trend

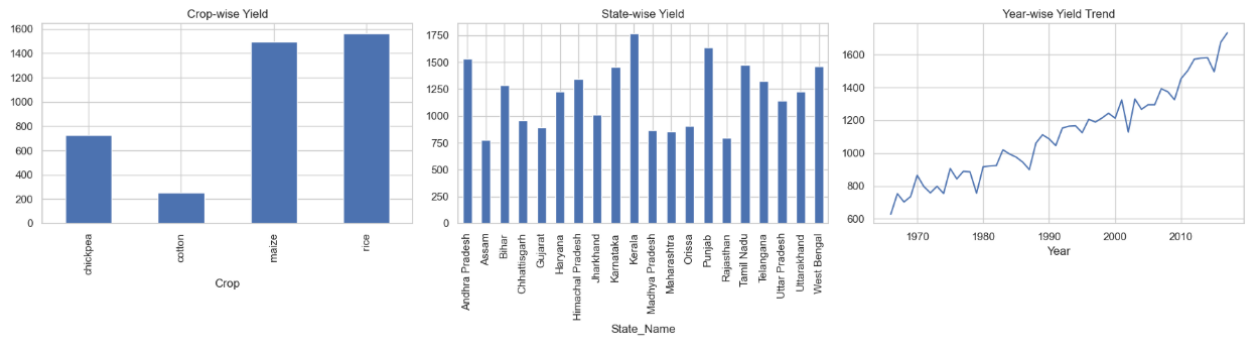
The year-wise trend shows fluctuations in crop yield over time. While some years show higher productivity due to favorable weather conditions, others show lower yield due to adverse climatic factors such as drought or irregular rainfall.

Insight: Crop yield is influenced by temporal factors, and yearly variations reflect the impact of changing environmental and climatic conditions.

Conclusion

Trend analysis highlights that crop yield is not constant and varies across crops, regions, and time. These insights are important for developing predictive models and making informed decisions for agricultural planning and resource optimization.

```
: # identify patterns/trend:
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
df.groupby("Crop")["Yield_kg_per_ha"].mean().plot(
    kind="bar", ax=axes[0], title="Crop-wise Yield"
)
axes[0].tick_params(axis='x', rotation=90)
df.groupby("State_Name")["Yield_kg_per_ha"].mean().plot(
    kind="bar", ax=axes[1], title="State-wise Yield"
)
axes[1].tick_params(axis='x', rotation=90)
df.groupby("Year")["Yield_kg_per_ha"].mean().plot(
    ax=axes[2], title="Year-wise Yield Trend"
)
plt.tight_layout()
plt.show()
```

4.5 FEATURE ENGINEERING

Feature engineering is a critical step in the machine learning pipeline that focuses on creating new features and transforming existing ones to improve the performance of predictive models. In this project, feature engineering was carried out to make the dataset more informative and suitable for modeling.

New features such as **Total Fertilizer** were created by combining individual components like Nitrogen, Phosphorus, and Potassium, which helped in simplifying the dataset while retaining essential information. Additionally, categorical variables such as state and crop type were converted into numerical form using encoding techniques, enabling them to be effectively used by machine learning algorithms.

These transformations ensured that the data was structured in a way that models could easily learn patterns and relationships.

Furthermore, feature engineering was also used to address important challenges such as **multicollinearity** and **data leakage**. Since fertilizer-related variables showed a very high correlation with the target variable (yield), directly using them could lead to overfitting and unrealistic model performance.

To overcome this, redundant or highly correlated features were either removed or combined into a single representative feature. This helped in reducing model complexity and improving generalization.

Overall, feature engineering enhanced the quality of the dataset, leading to more accurate, stable, and reliable predictions in the machine learning model.

4.5.1 Creating a New Feature: Total Fertilizer

In the feature engineering stage, a new feature called **Total Fertilizer** was created by combining the individual fertilizer components, namely Nitrogen (N), Phosphorus (P), and Potassium

(K). These three nutrients are the primary fertilizers used in agriculture and collectively influence crop growth and yield. Instead of using them separately, they were aggregated into a single feature representing the overall fertilizer input per hectare. This transformation helped in simplifying the dataset while still capturing the essential information about fertilizer usage.

The creation of the **Total_Fertilizer** feature was also important for reducing multicollinearity, as the individual N, P, and K variables were highly correlated with each other and with the target variable (yield). Using them separately could lead to overfitting and unstable model performance. By combining them into a single variable, the model becomes more robust and easier to interpret. This new feature significantly contributed to improving the predictive performance and generalization ability of the machine learning models used in the project.

4.5.2 Checking Skewness of Data

In this step, the distribution of numerical features was analyzed to identify skewness, which refers to the asymmetry in the distribution of data. Skewness was examined using histograms and distribution plots for variables such as **Yield_kg_per_ha**, **Total_Fertilizer**, **Rainfall_mm**, and **Area_ha**.

It was observed that several features, especially the target variable (Yield) and fertilizer-related variables, exhibited positive (right) skewness, where most values are concentrated on the lower side with a long tail extending towards higher values.

This skewness can negatively impact machine learning models by making them sensitive to extreme values and reducing prediction accuracy. Therefore, identifying skewness is important to ensure that the data follows a more balanced distribution. Based on this analysis, appropriate transformation techniques were applied to reduce skewness and improve model performance.

Handling Skewness using Log Transformation

Log transformation is a commonly used technique to reduce positive skewness in data. In this method, the logarithm of the feature values is taken, which compresses the range of large values and brings the distribution closer to normal.

In this project, log transformation was applied to skewed variables such as **Yield_kg_per_ha** and **Total_Fertilizer**, where extreme high values were present. After applying the transformation, the distribution became more symmetric, and the effect of outliers was reduced.

Mathematically, log transformation can be written as:

$$X' = \log(X) \quad (10)$$

This helped in stabilizing variance and improving the performance of linear models such as

Lasso and Ridge regression. However, log transformation can only be applied to positive values.

Handling Skewness using Box-Cox Transformation

The Box-Cox transformation is a more advanced technique used to normalize skewed data. Unlike log transformation, it can automatically determine the optimal transformation parameter (λ) to make the data distribution as close to normal as possible.

The Box-Cox transformation is defined as:

$$X' = \begin{cases} \frac{X^\lambda - 1}{\lambda}, & \text{if } \lambda \neq 0 \\ \log(X), & \text{if } \lambda = 0 \end{cases} \quad (11)$$

In this project, Box-Cox transformation was applied to selected numerical features that exhibited strong skewness. This method reduced asymmetry and improved the overall distribution.

The Box-Cox transformation is more flexible and powerful compared to log transformation, as it adapts to the data automatically. However, similar to log transformation, it requires the data to be strictly positive.

```
# checking the skewness of the columns
df.skew(numeric_only=True)
```

```
# handling the skewed data
# Log transforms
df['Area_ha'] = np.log1p(df['Area_ha'])
df['Rainfall_mm'] = np.log1p(df['Rainfall_mm'])
df['P_req_kg_per_ha'] = np.log1p(df['P_req_kg_per_ha'])
# Box-Cox transforms
df['N_req_kg_per_ha'], _ = boxcox(df['N_req_kg_per_ha'] + 1)
df['K_req_kg_per_ha'], _ = boxcox(df['K_req_kg_per_ha'] + 1)
df['Total_Fertilizer'], _ = boxcox(df['Total_Fertilizer'] + 1)
```

4.5.3 Checking Linearity Using Regplot

The linearity between fertilizer and crop yield was improved by applying **log transformation** to both variables. The transformed plot shows a strong linear relationship, with data points closely aligned along the regression line.

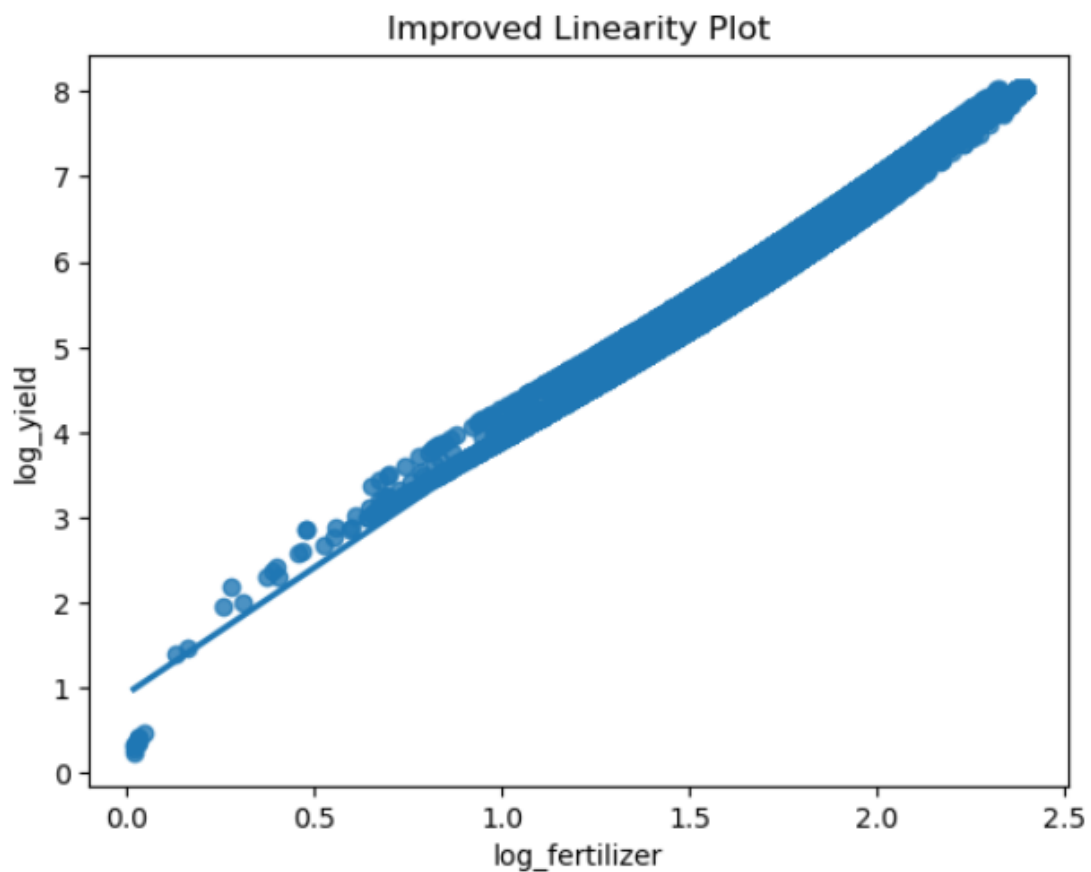
Dist_Code	1.364890
Year	0.000548
State_Code	0.397943
Area_ha	3.199830
Yield_kg_per_ha	0.937961
N_req_kg_per_ha	1.045239
P_req_kg_per_ha	0.994732
K_req_kg_per_ha	1.005970
Total_N_kg	4.713511
Total_P_kg	4.742728
Total_K_kg	4.817770
Temperature_C	0.369338
Humidity_percent	0.217724
pH	-0.273788
Rainfall_mm	0.594472
Wind_Speed_m_s	0.182468
Solar_Radiation_MJ_m2_day	0.182468
Total_Fertilizer	1.007959
dtype:	float64

(a) Before Transformation

Dist_Code	1.364890
Year	0.000548
State_Code	0.397943
Area_ha	-0.609946
Yield_kg_per_ha	0.937961
N_req_kg_per_ha	-0.034608
P_req_kg_per_ha	-0.309002
K_req_kg_per_ha	-0.025579
Total_N_kg	4.713511
Total_P_kg	4.742728
Total_K_kg	4.817770
Temperature_C	0.369338
Humidity_percent	0.217724
pH	-0.273788
Rainfall_mm	0.415431
Wind_Speed_m_s	0.182468
Solar_Radiation_MJ_m2_day	0.182468
Total_Fertilizer	-0.038329
dtype:	float64

(b) After Transformation

Comparison of Data Distribution



4.5.4 Detecting And Dropping Highly Correlated Features

In this step, the dataset was analyzed to identify highly correlated features, which can lead to multicollinearity—a situation where two or more independent variables are strongly related to each other. Multicollinearity can negatively affect machine learning models, especially linear models, by making coefficient estimates unstable and reducing the interpretability of the model.

To detect such relationships, a correlation matrix was generated and visualized using a heatmap, which helped in identifying pairs of features with very high correlation values (typically greater than 0.8 or 0.9). During the analysis, it was observed that fertilizer-related variables such as Nitrogen (N), Phosphorus (P), and Potassium (K) showed extremely high correlation with each other and also with the target variable (Yield kg per ha). This indicated redundancy in the information provided by these features and also raised the issue of potential data leakage.

To address this problem, highly correlated features were either removed or combined into a single meaningful feature such as **Total Fertilizer**, which retained the overall fertilizer information while reducing redundancy among input variables. This step helped in reducing unnecessary complexity and improving the quality of features provided to the machine learning models.

Removing highly correlated features improves the reliability and efficiency of the predictive model by ensuring that each selected feature contributes unique information. Since multiple features carrying similar information can increase model complexity without improving performance, feature reduction helps create a simpler and more generalized model.

In this project, correlation analysis played an important role in selecting meaningful input variables before model training. By reducing multicollinearity and avoiding data leakage, the machine learning models were able to learn the actual relationships between environmental factors, resource usage, and crop yield, leading to better prediction accuracy and improved model interpretability.

Furthermore, removing highly correlated variables helps to improve the stability and robustness of the model during prediction. When several independent variables contain almost identical information, the model may assign unnecessary importance to those features, which can affect its ability to perform well on new unseen data. By selecting only the most relevant features, the model becomes less sensitive to noise and avoids overfitting.

This feature selection process also reduces computational complexity by decreasing the number of input variables used during training. As a result, the machine learning algorithms can train faster while maintaining better generalization capability. Therefore, handling highly correlated features is an important preprocessing step that ensures the development of an accurate, efficient, and interpretable crop yield prediction model.

4.5.5 Encoding Categorical Columns using One-Hot Encoding

In this step, categorical variables in the dataset were converted into a numerical format using one-hot encoding, which is essential for applying machine learning algorithms. Categorical features such as **State Name** and **Crop Type** represent qualitative information and cannot be directly used by most machine learning models, which require numerical input.

Therefore, one-hot encoding was applied to transform these categories into a set of binary (0 or 1) columns, where each column represents a unique category. For example, a column like **State Name** is converted into multiple columns such as **State_Name_Telangana**, **State_Name_Uttar_Pradesh**, etc., where a value of 1 indicates the presence of that category and 0 indicates its absence.

One-hot encoding ensures that no ordinal relationship is imposed between categories, unlike label encoding, which may assign numerical values that could mislead the model into interpreting a hierarchy. In this project, there is no natural ordering between different states or crops, so one-hot encoding is the most appropriate method.

This transformation allows the model to treat each category independently, ensuring that the categorical information is accurately represented without introducing bias.

During implementation, the categorical columns were identified using their data types and then encoded using methods such as `pd.get_dummies()`. This process increased the number of columns, as each unique category resulted in a new binary feature.

Although this increases dimensionality, it provides a more detailed representation of the data, allowing the model to capture location-specific and crop-specific variations in yield.

As a result of one-hot encoding, the dataset became fully numerical and suitable for machine learning models such as Linear Regression, Lasso, Ridge, and Random Forest. This step significantly improved the model's ability to learn patterns related to different states and crops, ultimately contributing to better prediction accuracy and overall model performance.

4.5.6 Scaling The Numerical Columns

Scaling of numerical features is an important preprocessing step in machine learning that ensures all variables are on a similar range. In this project, numerical features such as **Temperature_C**, **Rainfall_mm**, **Humidity_percent**, **pH**, **Area_ha**, **Solar_Radiation_MJ_m2_day**, and **Total_Fertilizer** had different units and value ranges.

For example, rainfall values may range in hundreds or thousands, while pH values typically range between 0 and 14. Such differences in scale can negatively impact model performance,

especially for algorithms like Linear Regression, Lasso, and Ridge, which are sensitive to feature magnitude.

To address this issue, feature scaling was applied to normalize the numerical variables. The main objective of scaling is to bring all features to a comparable range so that no single feature dominates the model due to its larger scale.

Without scaling, features with higher values (such as rainfall or area) may have a greater influence on the model compared to smaller-scale features (such as pH), even if they are not more important.

In this project, scaling techniques such as **Standardization (Z-score normalization)** were used. Standardization transforms the data so that it has a mean of 0 and a standard deviation of 1, using the formula:

$$Z = \frac{X - \mu}{\sigma} \quad (12)$$

where μ is the mean and σ is the standard deviation of the feature.

This transformation ensures that all features contribute equally to the model and improves convergence during training.

Scaling also played a crucial role in improving the performance of regularized models such as Lasso and Ridge regression, which penalize large coefficients. Without scaling, these models may assign incorrect importance to features simply due to their scale rather than their actual predictive power.

By scaling the data, the regularization process becomes more effective, leading to better generalization and reduced overfitting.

Overall, scaling the numerical features helped in stabilizing the learning process, improving model accuracy, and ensuring fair contribution of all variables. This step was particularly important in achieving better performance in linear models, which ultimately resulted in Lasso Regression emerging as the best-performing model in this project.

4.5.7 Checking The Variance Inflation Factor(VIF)

In this step, the **Variance Inflation Factor (VIF)** was used to detect the presence of multicollinearity among the numerical features in the dataset. Multicollinearity occurs when independent variables are highly correlated with each other, which can negatively affect the stability and interpretability of machine learning models, especially linear models such as Linear Regression, Lasso, and Ridge.

VIF measures how much the variance of a regression coefficient is inflated due to the presence of correlation among predictors. A VIF value greater than 5 or 10 generally indicates a high level of multicollinearity.

In this project, VIF was calculated for numerical features such as **Total_Fertilizer**, **Rainfall_mm**, **Temperature_C**, **Humidity_percent**, and **Area_ha** to identify redundant variables.

It was observed that some features, particularly fertilizer-related variables (before aggregation), had very high VIF values, indicating strong multicollinearity. This confirmed the earlier findings from correlation analysis, where features like Nitrogen, Phosphorus, and Potassium were highly correlated with each other.

To address this issue, highly correlated variables were either removed or combined into a single feature such as **Total_Fertilizer**, which helped in reducing redundancy.

After this adjustment, the VIF values for the remaining features decreased significantly, indicating improved independence among variables. This step ensured that the model coefficients became more stable and interpretable.

```
# checking the Variance Inflation Factor:
X = df.select_dtypes(include=['float64', 'int64'])
vif_data = pd.DataFrame()
vif_data["Feature"] = X.columns
vif_data["VIF"] = [variance_inflation_factor(X.values, i) for i in range(len(X.columns))]
print(vif_data)
```

	Feature	VIF
0	Dist_Code	1.135702
1	Year	1.247822
2	State_Code	1.097371
3	Area_ha	1.464501
4	Yield_kg_per_ha	12.986665
5	Total_N_kg	1.861418
6	Temperature_C	inf
7	Humidity_percent	inf
8	pH	inf
9	Rainfall_mm	inf
10	Wind_Speed_m_s	inf
11	Total_Fertilizer	16.484439

4.5.8 Automatic Selection of Relevant Numerical and Important Features for the Model

In this step, relevant numerical features and important variables were automatically selected from the dataset to build an efficient and accurate machine learning model. The dataset contains a mix of numerical and categorical variables, but not all features contribute equally to predicting the target variable (**Yield_kg_per_ha**).

Therefore, it is important to select only those features that have meaningful relationships with the target and remove irrelevant or redundant variables.

The first step involved identifying numerical columns using data type-based selection methods. These columns include features such as *Temperature_C*, **Rainfall_mm**, **Humidity_percent**, **pH**, **Area_ha**, **Solar_Radiation_MJ_m2_day**, and **Total_Fertilizer**, which represent measurable quantities and are essential for modeling.

At the same time, unnecessary columns or those that could cause data leakage were excluded from the feature set to ensure fair model training.

Next, feature selection techniques such as correlation analysis and statistical methods (e.g., *SelectKBest*) were used to automatically identify the most important features. These methods evaluate the relationship between each feature and the target variable and select only those with the highest predictive power.

By doing this, the dimensionality of the dataset was reduced, and only the most relevant features were retained for model training.

As a result, the model was trained using a refined set of features that are both informative and non-redundant. This improved the model's performance, reduced computational complexity, and helped prevent overfitting.

Overall, automatic feature selection played a key role in building a more efficient and reliable machine learning model for crop yield prediction.

4.6 MODEL DEVELOPMENT

4.6.1 Train-Test Split

The initial step in model development is preparing the dataset for analysis by separating it into independent variables (features) and the target variable. In this project, the target variable is **Yield_kg_per_ha**, which represents the crop yield per hectare.

The remaining columns, including environmental, soil, and agricultural features, are treated as independent variables. This separation is essential because the model learns patterns from the input features to predict the target variable.

After defining the input (X) and output (y), the dataset is divided into training and testing sets. The training dataset is used to train the machine learning model, allowing it to learn relationships between features and the target variable.

The testing dataset is used to evaluate the performance of the trained model on unseen data, ensuring that the model generalizes well and does not simply memorize the training data.

In this project, the dataset was split into 80% training data and 20% testing data. This ratio ensures that the model has sufficient data to learn patterns while also reserving enough data for reliable evaluation.

The splitting was performed randomly using a fixed random state to ensure reproducibility of results.

Overall, the train-test split is a crucial step in model development as it helps in assessing the model's performance, detecting overfitting, and ensuring that the predictions are reliable and applicable to real-world scenarios.

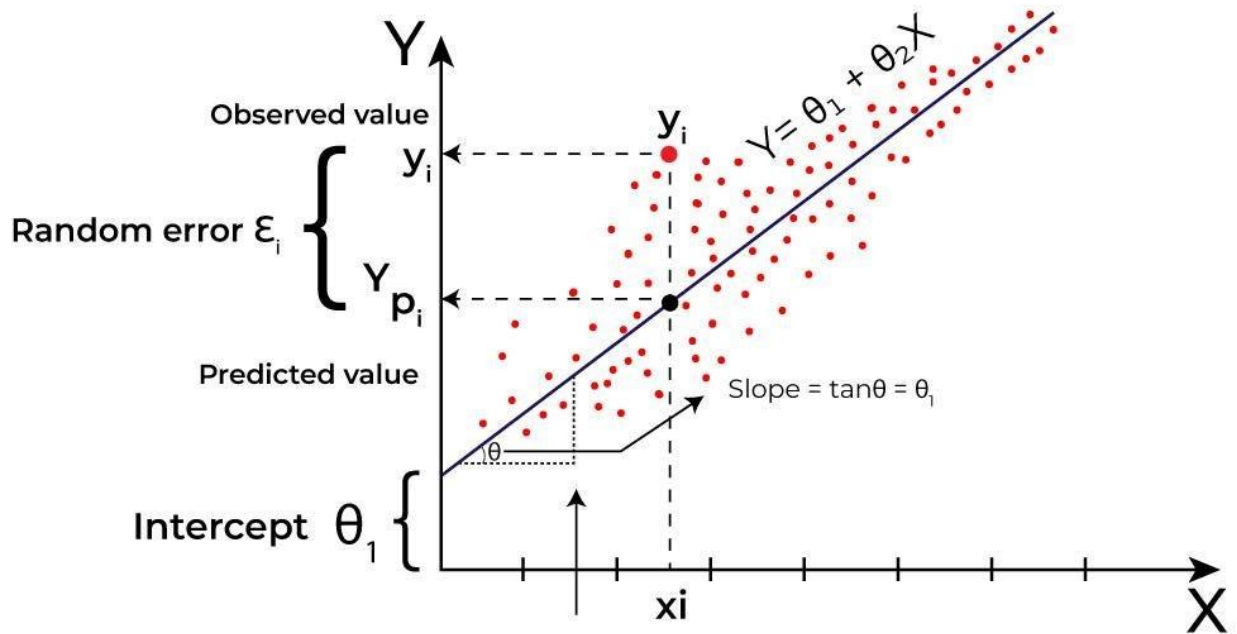
```
: # Define Features & Target
X = df.drop(['Yield_kg_per_ha'], axis=1)
y = df['Yield_kg_per_ha']
# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

4.6.2 Model Trainig

LINEAR REGRESSION MODEL

Linear Regression is a supervised machine learning algorithm used to model the relationship between a dependent variable and one or more independent variables. In this project, Linear Regression is used to predict crop yield (**Yield_kg_per_ha**) based on input features such as Temperature, Rainfall, Humidity, Soil pH, Area, and Fertilizer usage. The main idea of Linear

Regression is to find a best-fit line (or hyperplane in multiple dimensions) that represents the relationship between input features and the target variable. This line is chosen such that the difference between the actual values and predicted values is minimized. This difference is called the error, and the model minimizes it using a method known as **Least Squares**.



Mathematically, the model is represented as:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n \quad (13)$$

Where:

- Y = Predicted Yield
- $X_1, X_2, \dots, X_n$ = Input features (Rainfall, Temperature, etc.)
- β_0 = Intercept
- $\beta_1, \beta_2, \dots$ = Coefficients (impact of each feature)

How Linear Regression Works in Our Project

- The model takes environmental and agricultural features as input
- It learns how each feature affects crop yield
- It assigns weights (coefficients) to each feature
- It predicts yield for new data using the learned equation

For example:

- If rainfall increases, yield may increase
- If temperature is too high, yield may decrease

The model captures these relationships in a linear form.

Key Idea: Best Fit Line

The goal of Linear Regression is to find a line that:

- Minimizes the distance between actual and predicted values
- Reduces overall error (Residuals)
- Provides the most accurate predictions

This is done using the Least Squares Method, which minimizes:

$$\sum (Y_{\text{actual}} - Y_{\text{predicted}})^2 \quad (14)$$

Assumptions of Linear Regression

1. Linearity

There should be a linear relationship between independent variables and the target variable.

2. Independence of Errors

Residuals should be independent of each other.

3. Homoscedasticity (Constant Variance)

The variance of errors should remain constant across all levels of input variables.

4. No Multicollinearity

Independent variables should not be highly correlated. In this project, N, P, K showed high correlation and were combined into **Total Fertilizer**.

5. Normality of Errors

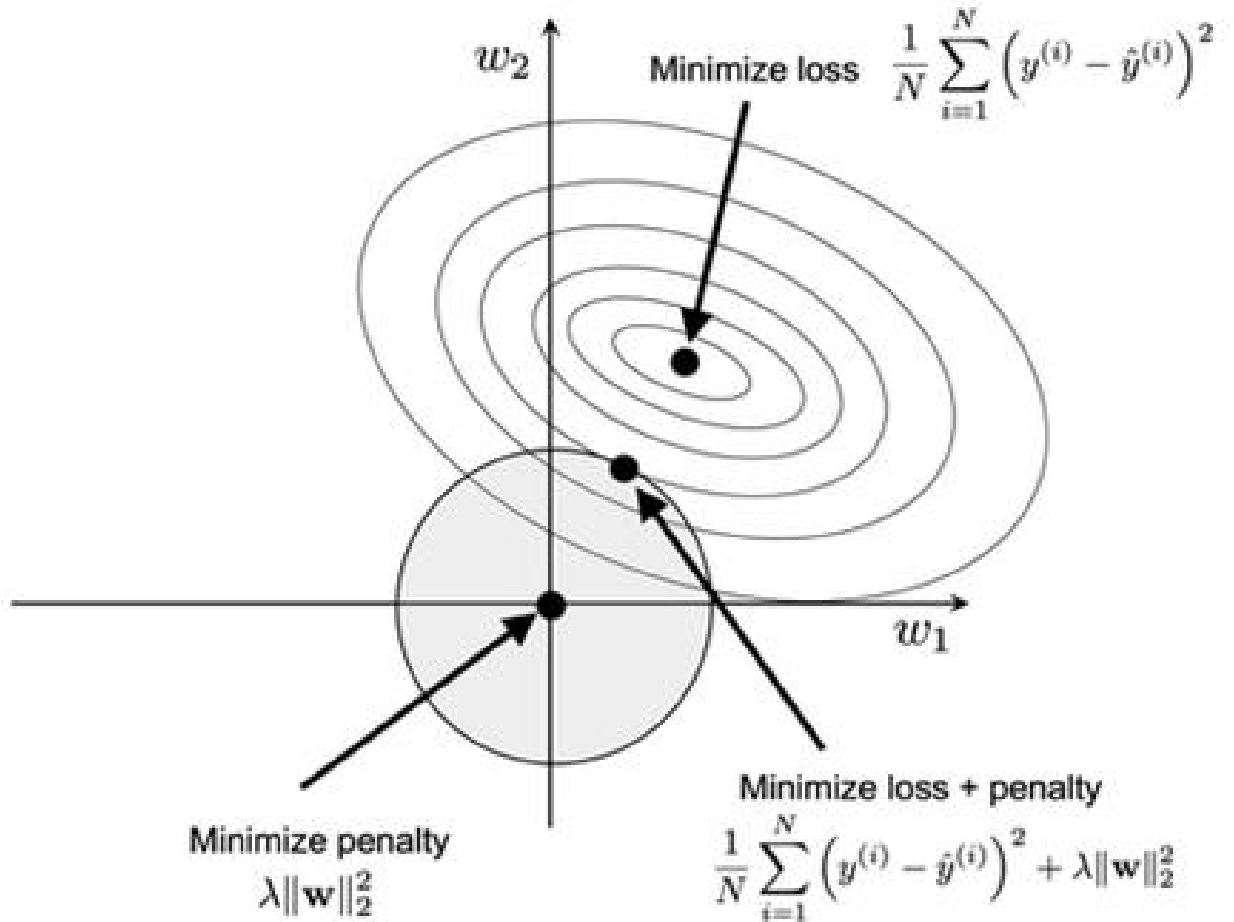
Residuals should be approximately normally distributed.

RIDGE REGRESSION MODEL

Ridge Regression is an extension of Linear Regression that is used to improve model performance when there is multicollinearity (high correlation among input features). In our crop

yield prediction project, several features such as environmental variables and fertilizer-related attributes showed correlation with each other, which can make the model unstable.

Ridge Regression addresses this issue by adding a penalty term (L_2 regularization) to the model, which controls the magnitude of the coefficients and prevents them from becoming too large.



Conceptually, Ridge Regression modifies the Linear Regression objective by minimizing not only the error between predicted and actual yield values but also the size of the coefficients. The model tries to find a balance between fitting the data well and keeping the coefficients small.

The objective function is given by:

$$\text{Minimize: } (Y_{\text{actual}} - Y_{\text{predicted}})^2 + \lambda \sum \beta_i^2 \quad (15)$$

where λ (lambda) is a tuning parameter that controls the strength of regularization. A higher value of λ results in stronger penalty, shrinking the coefficients more aggressively.

In the context of our project, this helps in reducing the effect of highly correlated variables such

as rainfall, humidity, and fertilizer, leading to a more stable and reliable model.

In our crop yield dataset, Ridge Regression was particularly useful because it handled the correlation between features like **Total_Fertilizer**, **Rainfall**, and **Temperature** effectively. Instead of removing features completely (as in Lasso), Ridge keeps all features but reduces their impact proportionally.

This ensures that important agricultural factors are not lost while still controlling overfitting. As a result, the model becomes more robust and performs well on unseen data.

Another important aspect of Ridge Regression is that it improves the generalization ability of the model. In our project, after preprocessing and feature scaling, Ridge Regression produced results very close to Lasso Regression, with high R^2 and low error values.

This indicates that the model was able to capture the underlying relationships between environmental factors and crop yield while avoiding overfitting. Although Ridge did not outperform Lasso, it provided stable and consistent predictions.

Overall, Ridge Regression played an important role in our project by handling multicollinearity, stabilizing the model, and improving prediction reliability. It demonstrated that regularization techniques are essential when dealing with real-world datasets where features are often correlated.

LASSO REGRESSION MODEL

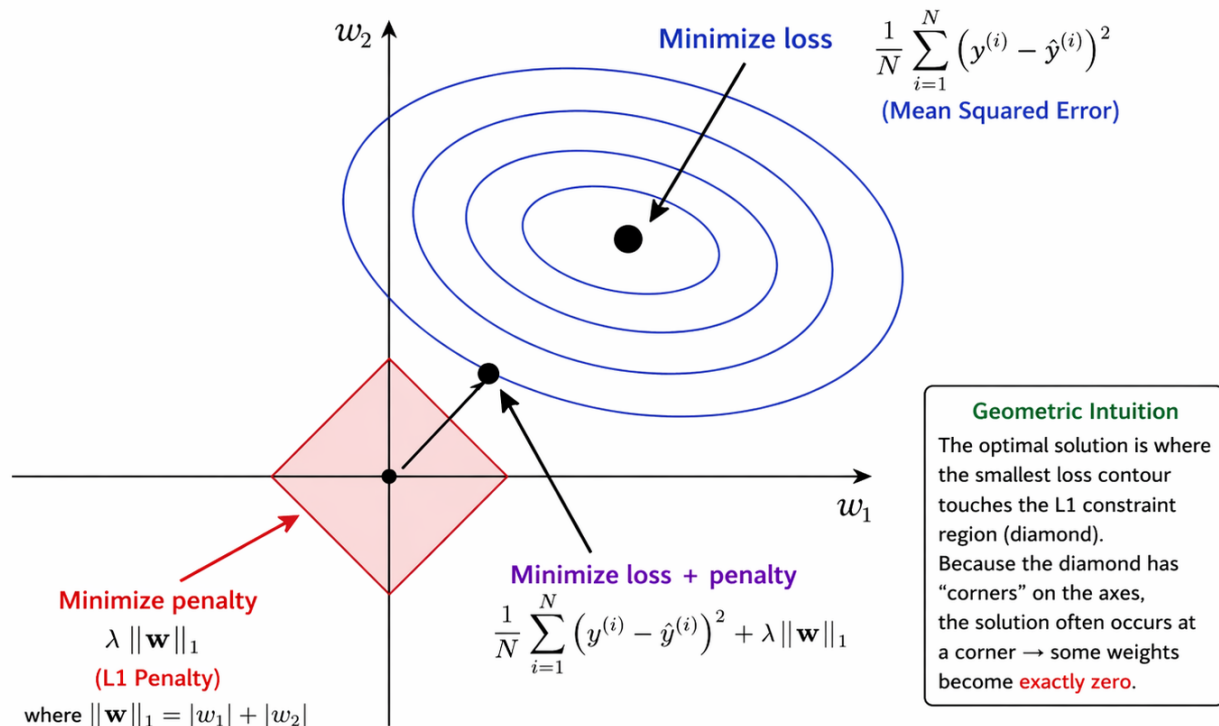
Lasso Regression (Least Absolute Shrinkage and Selection Operator) is an advanced form of linear regression that improves model performance by adding a regularization term (L_1 penalty) to the loss function. In a standard linear regression model, all features are used equally to predict the target variable, which in our case is **Yield_kg_per_ha**.

However, in real-world datasets like ours, not all features contribute equally to predicting crop yield. Some variables may be less important or even introduce noise. Lasso Regression addresses this problem by penalizing the absolute size of the coefficients, effectively reducing the impact of less important features.

Mathematically, Lasso modifies the cost function by adding a penalty term:

$$\text{Loss} = \text{RSS} + \lambda \sum_{i=1}^n |\beta_i| \quad (16)$$

LASSO REGRESSION (L1 Regularization)



Key Takeaways

- Lasso adds an L1 penalty (sum of absolute weights) to the loss.
- The L1 constraint region is a diamond.
- The optimal point lies on the boundary where the contour first touches the diamond.
- Many weights become exactly zero $\rightarrow$ Lasso performs **automatic feature selection (sparse model)**.

where RSS (Residual Sum of Squares) represents the prediction error, and λ (lambda) is the regularization parameter that controls the strength of the penalty.

As the value of λ increases, the model forces more coefficients to shrink towards zero. Unlike Ridge Regression, which only reduces the magnitude of coefficients, Lasso can make some coefficients exactly zero. This means it performs automatic feature selection by removing irrelevant variables from the model.

In the context of our project, the dataset contains multiple features such as Temperature, **Rain-fall**, **Humidity**, **pH**, **Area**, **Solar Radiation**, and **Total Fertilizer**, along with encoded categorical variables like state and crop type. Some of these features may have weak or redundant relationships with yield.

Lasso Regression helped in identifying and eliminating such less important features, retaining only those variables that significantly influence crop yield. This was especially useful after handling multicollinearity and removing highly correlated fertilizer variables.

Another important advantage of Lasso in this project is its ability to reduce overfitting. Since the dataset contains a large number of features after one-hot encoding, there is a risk

that the model may become too complex and start memorizing patterns instead of generalizing them.

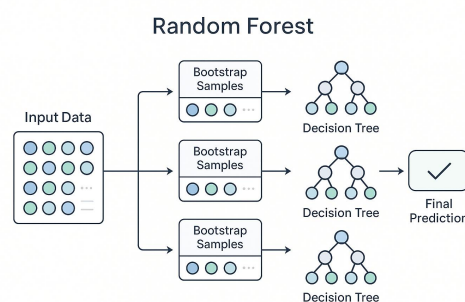
Lasso controls this complexity by shrinking coefficients, resulting in a simpler and more robust model. This is why, after proper preprocessing and feature engineering, Lasso Regression achieved the best performance ($R^2 \approx 0.93$) among all models in this project.

Conceptually, Lasso Regression works like a filter that keeps only the most relevant information and removes unnecessary noise. It ensures that the model focuses on the strongest predictors of crop yield while ignoring less useful variables.

Therefore, Lasso Regression was selected as the final model in this project due to its balance between accuracy, simplicity, and generalization ability.

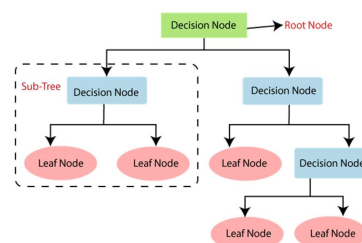
RANDOM FOREST REGRESSION MODEL

Random Forest Regression is an advanced ensemble learning algorithm that combines multiple decision trees to improve prediction accuracy and reduce overfitting. Unlike a single decision tree, which may memorize patterns from the training data, Random Forest builds many trees and aggregates their predictions to produce a more stable and reliable result.



(a) Bootstrap sampling

How Random Forest works?



(b) Node split

Random Forest

In the context of our project, Random Forest was used to predict crop yield (*Yield.kg_per_ha*) based on features such as temperature, rainfall, humidity, soil pH, area, and fertilizer usage. Since agricultural data often contains complex and non-linear relationships, Random Forest is well-suited to capture these patterns effectively.

Conceptually, Random Forest works by creating multiple decision trees using different subsets of the data. This process is called **bootstrap sampling**, where each tree is trained on a randomly selected subset of the dataset (with replacement). Additionally, at each split in a tree, only a random subset of features is considered.

This introduces diversity among the trees and ensures that each tree learns slightly different patterns. In our project, some trees may focus more on rainfall and temperature, while others may give more importance to fertilizer or soil conditions.

Each individual decision tree in the forest makes its own prediction for crop yield. These trees split the data based on conditions such as “Is rainfall greater than a certain value?” or “Is temperature within an optimal range?” At the leaf nodes, each tree outputs a predicted yield value.

The final prediction of the Random Forest model is obtained by averaging the predictions of all individual trees. This averaging process reduces the effect of noise and prevents overfitting, making the model more robust compared to a single decision tree.

Another important advantage of Random Forest is its ability to handle non-linear relationships and feature interactions. For example, crop yield may depend on a combination of rainfall and fertilizer rather than each factor independently.

Random Forest can automatically capture such interactions without requiring explicit feature engineering. It also provides **feature importance scores**, which help in identifying the most influential variables affecting yield.

However, in this project, although Random Forest initially showed strong performance, after proper preprocessing and feature engineering, **Lasso Regression** outperformed it in terms of accuracy and generalization.

This indicates that the underlying relationships in the cleaned dataset became more linear, making simpler models more effective.

In conclusion, Random Forest Regression is a powerful and flexible model that is capable of capturing complex patterns in agricultural data. It played an important role in understanding non-linear relationships in our dataset and provided valuable insights, even though it was not selected as the final model.

4.7 MODEL EVALUATION

Model evaluation is a critical stage in the machine learning process that involves assessing the performance of a trained model using quantitative metrics. It helps in determining how well the model can generalize to unseen data and ensures that the predictions are accurate and reliable.

In regression problems, where the target variable is continuous, evaluation focuses on measuring the difference between actual and predicted values.

To evaluate the performance of regression models, several statistical metrics are used. These metrics provide insights into model accuracy, error magnitude, and consistency across different

datasets.

4.7.1 Performance Metrics

Mean Squared Error (MSE)

Mean Squared Error (MSE) is one of the most commonly used metrics for evaluating regression models. It measures the average of the squared differences between the actual values and the predicted values.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (17)$$

Where:

y_i = Actual value

$\hat{y}_i$ = Predicted value

n = Number of observations

MSE gives higher weight to larger errors because the differences are squared. However, the unit of MSE is not the same as the original data.

Root Mean Squared Error (RMSE)

Root Mean Squared Error (RMSE) is the square root of MSE and provides the error in the same unit as the target variable.

$$\text{RMSE} = \sqrt{\text{MSE}} \quad (18)$$

RMSE is easier to interpret and is sensitive to large errors (outliers).

R-Squared (R^2 Score)

R-squared (R^2) represents the proportion of variance in the dependent variable that is explained by the model.

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} \quad (19)$$

Where:

SS_{res} = Residual sum of squares

SS_{tot} = Total sum of squares

R^2 values range between 0 and 1. A higher value indicates better model performance.

Adjusted R-Squared

Adjusted R-squared is a modified version of R^2 that takes into account the number of predictors in the model.

$$\text{Adjusted } R^2 = 1 - \left(\frac{(1 - R^2)(n - 1)}{n - k - 1} \right) \quad (20)$$

Where:

n = Number of observations

k = Number of predictors

Unlike R^2 , adjusted R^2 increases only when the new feature improves the model.

Cross-Validation Score (CV Score)

Cross-validation is a technique used to evaluate model performance by dividing the dataset into multiple subsets (folds). The model is trained and tested multiple times across these folds.

$$\text{CV Score} = \frac{1}{k} \sum_{i=1}^k R_{\text{fold}_i}^2 \quad (21)$$

The average performance across all folds gives the final CV score. It provides a more robust estimate of model performance and helps reduce overfitting.

Model Comparison:

	Model	R2	Adjusted R2	MSE	RMSE	\
0	Linear Regression	0.931129	0.928721	46112.401549	214.737983	
1	Lasso	0.932076	0.929701	45477.965381	213.255634	
2	Ridge	0.932010	0.929632	45522.530462	213.360096	
3	Random Forest	0.856452	0.851432	96112.116370	310.019542	
	MAE	CV Score				
0	160.361214	0.930830				
1	158.883674	0.931306				
2	159.017802	0.931279				
3	221.935063	0.861372				

4.7.2 Checking Overfitting In The Fitted Model

In this step, overfitting was evaluated by comparing the model's performance on the training data and the testing data using the R^2 score.

Overfitting occurs when a model performs very well on training data but poorly on unseen data, indicating that it has memorized the training patterns instead of learning general relationships.

In this project, the train R^2 score (0.9325) and test R^2 score (0.9320) were almost identical, showing a negligible difference between them.

This indicates that the model is not overfitting and is able to generalize well to new data.

Such consistency between training and testing performance confirms that the model is stable, reliable, and suitable for real-world prediction tasks.

```
# checking the overfitting in the fitted model:
train_score = models["Lasso"].score(X_train_scaled, y_train)
test_score = models["Lasso"].score(X_test_scaled, y_test)
print("Train R2:", train_score)
print("Test R2 :", test_score)
```

```
Train R2: 0.9325062107381421
Test R2 : 0.9320764359804988
```

4.7.3 Best Model Selection

After evaluating multiple machine learning models using performance metrics such as R^2 , Adjusted R^2 , MSE, RMSE, MAE, and Cross-Validation score, **Lasso Regression** was selected as the best-performing model.

Lasso achieved the highest R^2 score of **0.932076** and Adjusted R^2 of **0.929701**, indicating strong explanatory power.

It also produced the lowest error values, with MSE = **45477.96**, RMSE = **213.25**, and MAE = **158.88**, showing high prediction accuracy.

Additionally, Lasso obtained the highest cross-validation score of **0.931306**, demonstrating excellent generalization ability.

Compared to Linear Regression, Ridge, and Random Forest, Lasso consistently performed better across all evaluation metrics.

Therefore, based on both accuracy and stability, **Lasso Regression** was selected as the final model for crop yield prediction.

4.8 OPERATIONAL RESEARCH (OPTIMIZATION MODEL)

Operational Research (OR) techniques were applied in this project to determine the optimal allocation of agricultural resources, specifically fertilizer and water, in order to maximize profit.

While the machine learning model predicts crop yield, the OR model uses this prediction to make optimal decisions under given constraints.

This integration of machine learning and operational research enables a complete decision-support system for efficient farming.

4.8.1 Linking Machine Learning With Optimization

The first step in building the optimization model involved linking the machine learning predictions with a mathematical formulation suitable for linear programming.

Since advanced machine learning models such as **Lasso Regression** are non-linear and complex in nature, they cannot be directly embedded into a linear optimization model.

To overcome this limitation, a simplified linear approximation of the relationship between crop yield and key controllable variables was developed. A linear regression model was trained using important features such as **Total Fertilizer** and *Rainfall mm* (used as a proxy for water availability), which are directly related to farming decisions.

The resulting linear equation is given as:

$$\text{Yield} = aF + bW + c \quad (22)$$

where F (fertilizer) and W (water) are the decision variables, and a , b , and c are coefficients estimated from the data that represent the contribution of each factor to crop yield.

This transformation is critical because it converts the prediction problem into a linear mathematical form that can be used in optimization.

By doing so, the model can efficiently determine the optimal levels of fertilizer and water that maximize profit while satisfying practical constraints such as budget limits and resource availability.

4.8.2 Adjustment Of Coefficients

To ensure realistic agricultural values, the coefficients were adjusted within practical limits:

Fertilizer effect (a) = **10**

Water effect (b) = **0.5**

Base yield (c) = **500**

This step ensured that the yield values remain within a realistic range and the optimization model produces meaningful results.

4.8.3 Decision Variables

The optimization model uses the following decision variables:

F : Amount of fertilizer to be used (kg per hectare)

W : Amount of water to be used (mm)

These variables are the quantities that need to be optimized.

4.8.4 Objective Function

The objective of the model is to maximize profit, defined as:

$$\text{Profit} = (\text{Price} \times \text{Yield} \times \text{Area}) - \text{Cost}_F \times F - \text{Cost}_W \times W \quad (23)$$

Substituting the yield function:

$$\text{Maximize } Z = \text{Price} \times (aF + bW + c) \times \text{Area} - \text{Cost}_F \times F - \text{Cost}_W \times W \quad (24)$$

Where:

Price = 20 per kg yield

Cost_F = 2 per kg fertilizer

Cost_W = 1 per mm water

Area = 1 hectare

4.8.5 Constraints

The optimization model is subject to real-world constraints:

Resource Constraints

$$F \leq 150, \quad W \leq 1000 \quad (25)$$

Minimum Requirement Constraints

$$F \geq 50, \quad W \geq 300 \quad (26)$$

Budget Constraint

$$2F + W \leq 10000 \quad (27)$$

These constraints ensure that the solution is practical and feasible within available resources.

4.8.6 Optimization Method

In this project, the resource optimization problem was formulated as a **Linear Programming (LP)** model. Linear programming is a mathematical optimization technique used to find the best possible outcome (such as maximum profit or minimum cost) under a set of linear constraints.

It is suitable for this problem because both the objective function (profit) and the constraints (resource limits, budget, etc.) can be expressed as linear relationships in terms of the decision variables.

The LP model was implemented and solved using the PuLP library in Python, which provides an efficient framework to define variables, objective functions, and constraints, and compute the optimal solution.

Mathematical Formulation

Decision Variables

Let:

$$F = \text{Fertilizer (kg per hectare)}, \quad W = \text{Water (mm)} \quad (28)$$

Objective Function

The goal is to maximize profit:

$$\text{Maximize } Z = P \cdot (aF + bW + c) \cdot A - C_F \cdot F - C_W \cdot W \quad (29)$$

Where:

P = Price per unit yield

a, b, c = coefficients from regression

A = Area (hectares)

C_F = Cost of fertilizer

C_W = Cost of water

This equation represents total revenue minus total cost.

Constraints

The model is subject to the following linear constraints:

Fertilizer Limit

$$F \leq F_{\max} \quad (30)$$

Water Limit

$$W \leq W_{\max} \quad (31)$$

Minimum Requirement Constraints

$$F \geq F_{\min}, \quad W \geq W_{\min} \quad (32)$$

Budget Constraint

$$C_F \cdot F + C_W \cdot W \leq \text{Budget} \quad (33)$$

Nature of Linear Programming

All equations in the model are linear, meaning:

- Variables appear only in the first degree (no squares or products like FW)
- Relationships are proportional and additive
- The feasible region formed by constraints is convex

Because of this structure, linear programming guarantees that the optimal solution lies at one of the corner points of the feasible region.

Solution Approach

The PuLP solver applies optimization techniques such as the **Simplex method** to systematically evaluate feasible solutions and identify the one that maximizes the objective function.

The solver ensures that all constraints are satisfied while achieving the highest possible profit.

4.8.7 Optimal Solution

After solving the linear programming model, the optimal values for the decision variables were obtained as **150 kg of fertilizer** and **1000 mm of water**.

These values represent the best possible allocation of resources within the given constraints, such as budget limits, minimum requirements, and maximum availability.

The result indicates that both fertilizer and water are used at their upper allowable limits, suggesting that increasing these inputs contributes positively to crop yield and overall profitability.

This outcome reflects the strong influence of these resources on yield, as captured by the model coefficients.

The optimal solution ensures that all constraints are satisfied while achieving the most efficient use of available resources.

4.8.8 Maximum Profit

Using the optimal values of fertilizer and water, the model achieved a maximum profit of **48,700**.

This value represents the highest possible profit that can be obtained under the defined conditions, considering both the revenue generated from crop yield and the costs associated with resource usage.

The profit is calculated by subtracting the total cost of fertilizer and water from the total revenue obtained through yield production.

This result demonstrates the effectiveness of the optimization model in balancing input costs and output gains.

It also highlights how proper resource allocation can significantly improve economic outcomes in agriculture, making the model a valuable tool for decision-making.

output:optimal solution

Adjusted Coefficients:

a (Fertilizer): 10

b (Water): 0.5

c (Base Yield): 500

Status: Optimal

♦ Optimal Solution:

Fertilizer (kg): 150.0

Water (mm): 1000.0

🏆 Maximum Profit: 48700.0

5. RESULTS AND DISCUSSION

This section presents the outcomes of the machine learning and optimization models developed for crop yield prediction and resource allocation. The results are analyzed to understand model performance, interpret key findings, and evaluate the effectiveness of the proposed system.

5.1 Machine Learning Results

Multiple regression models were trained and evaluated, including Linear Regression, Lasso Regression, Ridge Regression, and Random Forest Regressor. The models were assessed using performance metrics such as R^2 , Adjusted R^2 , MSE, RMSE, MAE, and Cross-validation score.

Among all the models, **Lasso Regression** achieved the best performance, with an R^2 score of approximately **0.932**, indicating that the model explains about 93% of the variance in crop yield.

It also recorded the lowest error values (RMSE $\approx$ **213**, MAE $\approx$ **158**) and the highest cross-validation score (**0.931**), demonstrating strong generalization ability.

Linear Regression and Ridge Regression showed similar performance but were slightly less accurate compared to Lasso. The Random Forest model, although capable of capturing non-linear relationships, showed comparatively lower performance after preprocessing, indicating that the dataset became more suitable for linear modeling after transformations and feature engineering.

5.2 Model Validation And Assumptions

- **Linearity:** Improved significantly after applying log transformations to both fertilizer and yield.
- **Normality:** Residuals were approximately normally distributed, as confirmed by Q-Q plots.
- **Homoscedasticity:** Residual plots showed constant variance.
- **Multicollinearity:** Addressed using correlation analysis and VIF.
- **Overfitting:** Minimal difference between training and testing R^2 values.

These validations confirm that the final model is statistically sound and reliable.

5.3 Key Insights From EDA

- Fertilizer usage has a strong positive impact on crop yield.

- Rainfall contributes positively with moderate influence.
- Temperature showed a weaker and non-linear relationship.
- The fertilizer-yield relationship exhibited diminishing returns, corrected using log transformation.

5.4 Optimization Results

The optimization model was developed using Linear Programming to determine the optimal allocation of fertilizer and water.

Optimal Resource Allocation:

- Fertilizer = 150 kg
- Water = 1000 mm

Maximum Profit: 48,700

The results indicate that maximum profit is achieved when both resources are used at their upper limits within the defined constraints.

5.5 Discussion

The integration of machine learning and optimization provides a powerful framework for agricultural decision-making. The machine learning model accurately predicts crop yield, while the optimization model determines the best combination of inputs to maximize profit.

The results highlight that:

- Proper feature engineering and transformation significantly improve model performance.
- Linear models can outperform complex models when assumptions are satisfied.
- Resource optimization plays a crucial role in improving economic outcomes.

However, the optimization results also suggest that increasing inputs always increases profit within the model constraints. In real-world scenarios, factors such as environmental sustainability, soil health, and diminishing returns should also be considered.

6. CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project successfully developed an integrated system combining Machine Learning and Operational Research for crop yield prediction and optimal resource allocation. The machine learning models were trained using agricultural and environmental data to accurately predict crop yield.

Among the models evaluated, **Lasso Regression** emerged as the best-performing model, achieving high accuracy ($R^2 \approx 0.93$) with low error values and strong generalization ability.

Extensive data preprocessing, including handling missing values, outliers, skewness, and multicollinearity, significantly improved the quality of the dataset and model performance. Transformations such as log and Box-Cox helped satisfy key regression assumptions like linearity and normality.

Validation techniques, including cross-validation and residual analysis, confirmed that the model is stable and free from overfitting.

The project further extended beyond prediction by incorporating an optimization model using Linear Programming. This model determined the optimal levels of fertilizer and water required to maximize profit under real-world constraints such as budget and resource limits.

The results demonstrated that optimal resource allocation can significantly improve agricultural productivity and economic returns.

Overall, the proposed system provides a data-driven decision-support framework that not only predicts crop yield but also recommends optimal farming strategies. This makes the project highly practical and valuable for farmers, agricultural planners, and policymakers.

6.2 Future Work

Although the proposed system achieved strong results in crop yield prediction and resource optimization, several enhancements can be explored to further improve its accuracy, scalability, and real-world applicability.

1. **Inclusion of Soil and Geographic Data:** Future work can incorporate detailed soil properties (nutrients, texture, moisture) and geospatial data (latitude, longitude, satellite imagery) to improve prediction accuracy and provide location-specific recommendations.
2. **Use of Advanced Machine Learning Models:** More sophisticated models such as Deep Learning (Neural Networks), LSTM (for time-series), and ensemble boosting techniques

(LightGBM, CatBoost) can be applied to capture complex non-linear relationships more effectively.

3. **Multi-Crop and Crop Rotation Optimization:** The current model focuses on single-crop optimization. Future work can extend this to multi-crop planning and crop rotation strategies, enabling farmers to maximize long-term productivity and soil health.
4. **Time-Series and Seasonal Analysis:** Incorporating seasonal and temporal data such as monthly rainfall, temperature variations, and historical trends can improve predictions and allow dynamic decision-making across different seasons.
5. **Real-Time Decision Support System:** The model can be integrated with real-time data sources such as weather APIs, IoT sensors, and smart irrigation systems to provide live recommendations for farmers.
6. **Environmental Sustainability Constraints:** Future optimization models can include environmental constraints, such as limiting excessive fertilizer usage, reducing water consumption, and minimizing carbon footprint to promote sustainable farming.
7. **Uncertainty and Risk Modeling:** Agricultural systems are affected by uncertainties such as weather fluctuations and market prices. Future work can incorporate stochastic optimization or probabilistic models to handle uncertainty and risk.
8. **Economic and Market Integration:** The model can be enhanced by including market price fluctuations, demand forecasting, and supply chain factors, enabling farmers to make profit-driven decisions beyond production.
9. **Deployment as a Web or Mobile Application:** The system can be developed into a user-friendly web or mobile application where farmers can input parameters and receive predictions and optimization recommendations instantly.
10. **Integration with Government and Policy Systems:** The model can be extended to support agricultural policy planning, helping governments design better subsidy programs, irrigation policies, and resource allocation strategies.
11. **Large-Scale Deployment with Big Data:** With access to larger datasets, the model can be scaled using big data technologies and cloud platforms, enabling nationwide or global agricultural analysis.

7. REFERENCES

1. Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and Tensor-Flow* (2nd ed.). O'Reilly Media.
2. GeeksforGeeks. *Machine Learning and Python Tutorials*.
Available at: <https://www.geeksforgeeks.org>
3. Winston, W. L. (2004). *Operations Research: Applications and Algorithms* (4th ed.). Thomson Brooks/Cole.

End of Project Report
